BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC KINH TẾ TP. HỒ CHÍ MINH

TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ

UEH UNIVERSITY

BÁO CÁO ĐỒ ÁN MÔN HỌC

MÁY HỌC

ĐỒ ÁN MÔN HỌC

ỨNG DỤNG GIẢI THUẬT INCREMENTAL DBSCAN TRONG PHÂN CỤM DỮ LIỆU ĐỘNG

Thực nghiệm cập nhật cụm khi thêm, xóa điểm và đối chiếu với DBSCAN của scikit-learn.

Học phần: Máy học

Giảng viên: TS. Nguyễn An Tế

Lớp - Nhóm: 26C1INF50904401 - Nhóm 1

Thành viên: Nguyễn Hoàng Anh - MSSV: 31241021759

Lý Huỳnh Phương Anh - MSSV: 31241020523

Từ Thiên Gia Bảo - MSSV: 311221024417

Nguyễn Phạm Gia Bảo - MSSV: 31221023273

Hồ Ngọc Bảo - MSSV: 31231020324

TP. Hồ Chí Minh, tháng 11 năm 2026

LỜI MỞ ĐẦU

Đề tài Incremental DBSCAN được giảng viên giao cho nhóm. Đề tài tập trung vào cách cập nhật kết quả phân cụm khi dữ liệu thay đổi theo thời gian. Nếu chạy lại toàn bộ DBSCAN sau mỗi lần thêm hoặc xóa điểm, chương trình phải xử lý lại cả những điểm không bị ảnh hưởng (Ester et al., 1998).

Trong báo cáo, nhóm giới thiệu DBSCAN và các bước cập nhật của Incremental DBSCAN, sau đó trình bày chương trình tự cài đặt theo hướng đối tượng. Các ví dụ 2D giúp theo dõi từng trường hợp thêm và xóa điểm; dữ liệu ATM và luồng giao dịch thẻ mô phỏng được dùng để minh họa, kiểm tra và so sánh kết quả.

Nhóm dùng Silhouette tự cài để nhận xét chất lượng cụm, đồng thời xem số cụm và tỷ lệ điểm nhiễu. Độ tương đương được kiểm tra bằng cách đối chiếu với DBSCAN của scikit-learn chạy lại trên dữ liệu sau cập nhật. Kết quả chỉ phản ánh các dữ liệu mô phỏng, tham số và số bước được trình bày trong báo cáo.

LỜI CẢM ƠN

Nhóm xin chân thành cảm ơn TS. Nguyễn An Tế đã hướng dẫn và góp ý trong quá trình học tập, thực hiện đồ án môn Máy học.

Nhóm cũng cảm ơn các thành viên đã cùng tìm hiểu tài liệu, xây dựng ví dụ, cài đặt chương trình, chuẩn bị dữ liệu và kiểm tra kết quả. Những công việc này giúp nhóm hiểu rõ hơn cách DBSCAN hình thành cụm và cách Incremental DBSCAN xử lý các thay đổi của dữ liệu.

Báo cáo và chương trình vẫn có thể còn thiếu sót. Nhóm mong nhận được góp ý để hiểu đúng hơn về thuật toán và cải thiện cách trình bày.

MỤC LỤC

LỜI MỞ ĐẦU i

LỜI CẢM ƠN ii

DANH MỤC BẢNG BIỂU vi

DANH MỤC HÌNH ẢNH vii

DANH MỤC TỪ VIẾT TẮT viii

1 TỔNG QUAN ĐỀ TÀI 1

1.1 Bối cảnh và lý do thực hiện đề tài . . . 1

1.2 Bài toán cần giải quyết . . . 1

1.3 Mục tiêu nghiên cứu . . . 1

1.4 Phạm vi nghiên cứu . . . 2

1.5 Phương pháp thực hiện . . . 2

1.6 Bố cục báo cáo . . . 2

2 CƠ SỞ LÝ THUYẾT 4

2.1 Tổng quan về phân cụm dữ liệu . . . 4

2.1.1 Khái niệm và mục tiêu của phân cụm dữ liệu . . . 4

2.1.2 Các phương pháp phân cụm phổ biến . . . 4

2.2 Thuật toán DBSCAN . . . 5

2.2.1 Các tham số Eps và MinPts . . . 5

2.2.2 Đối tượng lõi, đối tượng biên và nhiễu . . . 7

2.2.3 Nguyên lý mở rộng cụm . . . 8

2.2.4 Quy trình thực hiện . . . 9

2.2.5 Ví dụ minh họa DBSCAN từng bước . . . 11

2.2.6 Ưu điểm và hạn chế . . . 13

2.3 Thuật toán Incremental DBSCAN . . . 14

2.3.1 Nhu cầu cập nhật cụm khi dữ liệu thay đổi . . . 14

2.3.2 Vùng bị ảnh hưởng và tập điểm cần cập nhật . . . 15

2.3.3 Các trường hợp khi thêm điểm . . . 18

2.3.4 Các trường hợp khi xóa điểm . . . 22

2.3.5 Khác biệt giữa Incremental DBSCAN và chạy lại DBSCAN . . . 27
2.4 Các chỉ số đánh giá . . . 29
2.4.1 Adjusted Rand Index (ARI) . . . 29
2.4.2 Silhouette . . . 29
2.4.3 Thời gian chạy và bộ nhớ . . . 31
2.4.4 Số cụm và số điểm nhiễu . . . 31
2.5 Các thuật toán dùng để so sánh . . . 32
3 TỔNG QUAN BỘ DỮ LIỆU 34
3.1 Nguồn gốc và cách sử dụng dữ liệu . . . 34
3.2 Mô tả các bộ dữ liệu . . . 34
3.3 Tập ban đầu và các thao tác cập nhật . . . 35
3.4 Các chiều dữ liệu trong benchmark . . . 35
3.5 Giới hạn của dữ liệu . . . 36
4 TIỀN XỬ LÝ VÀ CÀI ĐẶT 37
4.1 Môi trường và thư viện sử dụng . . . 37
4.2 Khảo sát dữ liệu . . . 37
4.3 Làm sạch dữ liệu . . . 38
4.4 Chuẩn hóa dữ liệu . . . 38
4.5 Thiết lập tập ban đầu và luồng cập nhật . . . 39
4.6 Lựa chọn tham số eps và MinPts . . . 39
4.7 Thiết kế hướng đối tượng và sơ đồ lớp . . . 41
4.8 Cài đặt DBSCAN và Incremental DBSCAN . . . 43
4.9 Cài đặt đánh giá và trực quan hóa . . . 44
4.10 Kiểm thử tự động . . . 45
5 THỰC NGHIỆM VÀ ĐÁNH GIÁ 46
5.1 Thiết lập thực nghiệm . . . 46
5.2 Kết quả khi thêm điểm . . . 46
5.3 Kết quả khi xóa điểm . . . 48
5.4 Minh họa trên không gian 2D . . . 49

5.5 Đối chiếu với DBSCAN chạy lại 50
5.6 So sánh chất lượng cụm bằng Silhouette 50
5.7 Thời gian chạy và giới hạn đo đạc 51
5.8 Nhận xét chung 53
6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 54
6.1 Kết luận 54
6.2 Hạn chế của đề tài 54
6.3 Hướng phát triển 55
TÀI LIỆU THAM KHẢO 56
PHỤ LỤC 58
A - BẢNG PHÂN CÔNG CÔNG VIỆC 58
B - CÔNG BỐ VIỆC SỬ DỤNG AI 58
C - MÃ NGUỒN VÀ HƯỚNG DẪN CHẠY 58
D - KẾT QUẢ, HÌNH ẢNH BỔ SUNG 59

DANH MỤC BẢNG BIỂU

2.1 Lân cận ε và phân loại đối tượng trong ví dụ DBSCAN 11
2.2 Ảnh hưởng cục bộ của phép chèn hoặc xóa đối tượng 16
2.3 Các trường hợp cập nhật khi chèn đối tượng 19
2.4 Các trường hợp cập nhật khi xóa đối tượng 23
2.5 So sánh cách cập nhật kết quả phân cụm 28
2.6 So sánh đặc điểm của các thuật toán phân cụm 33

3.1 Các bộ dữ liệu được dùng trong đề tài 34
3.2 Các thuộc tính của luồng giao dịch thẻ 35

4.1 Thư viện và mục đích sử dụng 37
4.2 Kiểm tra ban đầu các file dữ liệu được dùng 38
4.3 Các cấu hình Eps và MinPts trong đồ án 40
4.4 Cấu hình thuật toán trong benchmark 41
4.5 Trách nhiệm của các lớp và phần hỗ trợ 43
4.6 Các tình huống kiểm thử chính của Incremental DBSCAN 45

5.1 Các cấu hình chính trong thực nghiệm 46
5.2 Kết quả các trường hợp INSERT 47
5.3 Kết quả các trường hợp DELETE 48
5.4 So sánh Silhouette trên dữ liệu ATM 2D 50
5.5 Silhouette trên các lát 2D, 3D và 4D của luồng thẻ 51
5.6 Thời gian cập nhật trên 20 thao tác 51
5.7 Thời gian fit ban đầu D0, trung vị qua 5 lượt (ms) 52

A.1 Phân công công việc và tỷ lệ đóng góp của thành viên 58

DANH MỤC HÌNH ẢNH

2.1 Minh họa các trường hợp cập nhật khi chèn đối tượng: Noise, Creation, Absorption và Merge 19
2.2 Minh họa các trường hợp cập nhật khi xóa đối tượng: Removal, Reduction, Split và Non-split 24

4.1 Sơ đồ các lớp chính trong chương trình 42

5.1 Các bước chèn điểm làm hai cụm nhập lại thành một cụm 47
5.2 Các bước xóa điểm cầu nối làm một cụm tách thành hai cụm 49
5.3 Tổng thời gian cập nhật qua 5 lượt benchmark 52

D.1 Một số trường hợp biên bổ sung của thao tác thêm và xóa điểm 60
D.2 Thang đo đặc trưng luồng thẻ trước và sau chuẩn hóa 61

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ
ARI	Adjusted Rand Index
BFS	Breadth-First Search
CSV	Comma-Separated Values
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
D0	Tập dữ liệu ban đầu trước các thao tác cập nhật
Eps	Bán kính lân cận dùng để xác định các điểm gần nhau
FIFO	First In, First Out
LIFO	Last In, First Out
HAC	Hierarchical Agglomerative Clustering
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
MinPts	Số điểm tối thiểu trong lân cận để một điểm là điểm lõi
OOP	Lập trình hướng đối tượng
OPTICS	Ordering Points To Identify the Clustering Structure

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh và lý do thực hiện đề tài

Trong thực tế, dữ liệu có thể thay đổi theo thời gian: có đối tượng mới được ghi nhận hoặc một số đối tượng cũ bị xóa. Nếu mỗi lần thay đổi đều chạy lại toàn bộ thuật toán phân cụm, chương trình sẽ phải xử lý lại cả những phần dữ liệu không bị ảnh hưởng (Ester et al., 1998).

Theo yêu cầu của đề tài, nhóm tìm hiểu Incremental DBSCAN. Thuật toán này dựa trên kết quả DBSCAN đã có để cập nhật các cụm sau khi dữ liệu thay đổi. Qua đề tài, nhóm muốn hiểu cách DBSCAN tạo cụm và cách cập nhật đó hoạt động trong các trường hợp thêm, xóa điểm (Ester et al., 1998).

1.2. Bài toán cần giải quyết

Đầu vào gồm một tập dữ liệu đã được phân cụm bằng DBSCAN. Sau đó, chương trình lần lượt thêm hoặc xóa một điểm. Với ε, MinPts và cách tính khoảng cách giữa các đối tượng bằng khoảng cách Euclidean được giữ cố định, chương trình cần cập nhật nhãn cụm, trạng thái lõi, biên và nhiễu cho dữ liệu mới.

Kết quả cập nhật được đối chiếu với DBSCAN chạy lại từ đầu trên cùng tập dữ liệu. Nhờ vậy, nhóm kiểm tra được thuật toán cập nhật có cho kết quả tương ứng hay không.

1.3. Mục tiêu nghiên cứu

Mục tiêu của đề tài là tự cài đặt và tìm hiểu Incremental DBSCAN qua một chương trình có thể chạy, minh họa và kiểm tra. Các việc chính gồm:

- Tìm hiểu cách DBSCAN xác định điểm lõi, điểm biên, điểm nhiễu và mở rộng cụm.
- Tự cài DBSCAN và Incremental DBSCAN bằng Python theo hướng đối tượng. Phần thuật toán của nhóm không gọi DBSCAN từ thư viện; NumPy và Pandas được dùng để hỗ trợ dữ liệu, còn scikit-learn được dùng làm đối chứng.
- Cài đặt hai thao tác cập nhật: thêm một điểm (INSERT) và xóa một điểm (DELETE).
- Kiểm tra kết quả cập nhật bằng cách đối chiếu với DBSCAN chạy lại trên dữ liệu sau cập nhật.
- Minh họa các bước trên ví dụ 2D và khảo sát dữ liệu nhiều chiều; đánh giá chất lượng

cụm chủ yếu bằng Silhouette, đồng thời xem số cụm, tỷ lệ nhiễu và thời gian chạy.

1.4. Phạm vi nghiên cứu

Đề tài tập trung vào một điểm được thêm hoặc xóa trong mỗi lần cập nhật. Trong các luồng thí nghiệm, nhóm giữ cố định cách tính khoảng cách giữa cách đối tượng bằng khoảng cách Euclidean, ε và MinPts. Việc tự chọn lại tham số sau mỗi cập nhật và cập nhật nhiều điểm cùng lúc không nằm trong phạm vi thực hiện.

Dữ liệu đều là dữ liệu mô phỏng, không phải giao dịch thật. Nhóm dùng một tập tọa độ ATM 2D để minh họa, một luồng giao dịch thẻ 5D cho các cập nhật và benchmark, cùng một số ví dụ nhỏ để trình bày từng trường hợp INSERT và DELETE. Các phép so sánh với DBSCAN, HDBSCAN và OPTICS của scikit-learn chỉ dùng phiên bản thư viện làm đối chứng; nhóm không tự cài lại ba đối chứng đó.

1.5. Phương pháp thực hiện

Nhóm thực hiện đề tài theo các bước sau:

1. Đọc tài liệu về DBSCAN và Incremental DBSCAN, sau đó tóm tắt các bước cần cài đặt.
2. Chuẩn bị dữ liệu mô phỏng: tập 2D, luồng thẻ và các ví dụ nhỏ có kết quả kỳ vọng.
3. Cài đặt DBSCAN và các thao tác INSERT, DELETE theo hướng đối tượng.
4. Chạy ví dụ minh họa, đối chiếu kết quả cập nhật với DBSCAN chạy lại, rồi đánh giá chất lượng cụm và thời gian trên các cấu hình đã chọn.

1.6. Bố cục báo cáo

Báo cáo gồm sáu chương:

- **Chương 1 - Tổng quan đề tài:** Giới thiệu bài toán, mục tiêu, phạm vi và cách nhóm thực hiện.
- **Chương 2 - Cơ sở lý thuyết:** Trình bày DBSCAN, Incremental DBSCAN, các bước cập nhật và các thuật toán, chỉ số dùng để đối chiếu.
- **Chương 3 - Tổng quan bộ dữ liệu:** Mô tả dữ liệu ATM, luồng thẻ và các ví dụ kiểm thử.
- **Chương 4 - Tiền xử lý và cài đặt:** Trình bày việc chuẩn bị dữ liệu, tham số và cách

tổ chức chương trình.

- **Chương 5 - Thực nghiệm và đánh giá:** Minh họa các cập nhật, kiểm tra kết quả và phân tích các phép so sánh.
- **Chương 6 - Kết luận và hướng phát triển:** Tóm tắt kết quả và nêu những điểm còn hạn chế.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về phân cụm dữ liệu

2.1.1 Khái niệm và mục tiêu của phân cụm dữ liệu

Phân cụm (Clustering) là một phương pháp học không giám sát (Unsupervised Learning). Phân cụm không sử dụng nhãn lớp có sẵn để xác định các nhóm mà tìm kiếm cấu trúc nhóm dựa trên đặc trưng của các đối tượng và tiêu chí phân cụm được lựa chọn (scikit-learn developers, n.d.-a).

Nhóm tập trung vào DBSCAN và Incremental DBSCAN, trong đó cụm được tìm qua mật độ và các kết nối giữa những điểm lõi. Thuật toán này cho phép những đối tượng không đáp ứng điều kiện thuộc cụm được xác định là nhiễu. Tuy nhiên, tập nhiễu không nhất thiết phải có phần tử nếu tất cả đối tượng đều được đưa vào các cụm (Ester et al., 1996).

2.1.2 Các phương pháp phân cụm phổ biến

Các thuật toán phân cụm có thể được phân biệt dựa trên cách xác định sự tương đồng và tổ chức các đối tượng thành cụm. Trong đó, ba loại phổ biến là phân cụm theo phân hoạch, phân cụm phân cấp và phân cụm dựa trên mật độ. Mỗi loại sử dụng một nguyên tắc hình thành cụm khác nhau, do đó có thể tạo ra những kết quả khác nhau khi áp dụng trên cùng một tập dữ liệu (scikit-learn developers, n.d.-a).

Phân cụm theo phân hoạch (Partitioning Clustering)

Phương pháp phân cụm theo phân hoạch chia dữ liệu thành số cụm đã chọn trước. K-means là ví dụ quen thuộc: mỗi điểm được gán cho tâm gần nhất, sau đó các tâm được cập nhật. Cách này cần khai báo số cụm và có thể không phù hợp với cụm cong hoặc có hình dạng phức tạp (Machine Learning Cơ Bản, 2017).

Phân cụm phân cấp (Hierarchical Clustering)

Phương pháp phân cụm phân cấp xây dựng cấu trúc thể hiện quan hệ giữa các cụm nhỏ và những cụm lớn được hình thành từ chúng. Phân cụm phân cấp kết tụ (HAC) bắt đầu bằng cách xem mỗi điểm là một cụm riêng, rồi gộp dần các cụm theo tiêu chí khoảng cách đã chọn. Kết quả thường được biểu diễn bằng cây để thấy thứ tự các cụm được gộp (scikit-learn developers, n.d.-a).

Phân cụm dựa trên mật độ (Density-Based Clustering)

Phương pháp dựa trên mật độ xét sự phân bố của các đối tượng trong những vùng

lân cận. Các vùng có đủ mật độ và có quan hệ kết nối với nhau được xác định thuộc cùng một cụm, còn các đối tượng không đáp ứng điều kiện tham gia cụm có thể được nhận diện là nhiễu. DBSCAN thuộc nhóm này, thuật toán có thể phát hiện các cụm có hình dạng không nhất thiết tập trung quanh một tâm và không yêu cầu khai báo trước số lượng cụm; kết quả phụ thuộc vào các tham số xác định vùng lân cận và ngưỡng mật độ (Ester et al., 1996).

2.2. Thuật toán DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) là thuật toán phân cụm dựa trên mật độ do Ester và cộng sự đề xuất. Thuật toán xét số lượng đối tượng trong vùng lân cận, mở rộng cụm từ các điểm lõi và đánh dấu các điểm không thuộc cụm là nhiễu (Ester et al., 1996).

Để thực hiện quá trình phân cụm, DBSCAN sử dụng hai tham số chính là ε (Eps) và MinPts. Hai tham số này quyết định điều kiện để một đối tượng có thể làm điểm bắt đầu mở rộng cụm, cũng như phạm vi các đối tượng được liên kết với nhau (Ester et al., 1996).

2.2.1 Các tham số Eps và MinPts

Bán kính lân cận ε (Eps)

ε (Eps) là ngưỡng khoảng cách dùng để xác định những đối tượng nằm trong lân cận của một đối tượng đang xét. Với tập dữ liệu D và hàm khoảng cách d, lân cận ε của đối tượng p, ký hiệu là $N_\varepsilon(p)$, được định nghĩa:

$$N_\varepsilon(p) = \{q \in D \mid d(p,q) \leq \varepsilon\}. \tag{2.1}$$

Theo định nghĩa này, một đối tượng q sẽ thuộc $N_\varepsilon(p)$ nếu khoảng cách từ p đến q không vượt quá ε. Những đối tượng có khoảng cách lớn hơn ε sẽ không được xem là lân cận trực tiếp của p.

Trong phạm vi cài đặt của nhóm, khoảng cách giữa hai đối tượng được tính bằng khoảng cách Euclid. Giả sử hai đối tượng p và q đều được mô tả bằng d giá trị số, khoảng cách giữa chúng được xác định như sau:

$$d(p,q) = \sqrt{\sum_{j=1}^{d} (p_j - q_j)^2}. \tag{2.2}$$

Trong đó, p_j và q_j lần lượt là giá trị thuộc tính thứ j của hai đối tượng, còn d là số chiều của dữ liệu. Công thức tính khoảng cách Euclid dựa trên độ chênh lệch giữa các thuộc tính tương ứng. Vì vậy, khi các thuộc tính có đơn vị hoặc thang đo khác nhau, những thuộc tính có giá trị số lớn có thể chi phối khoảng cách, từ đó ảnh hưởng đến kết quả xác định lân cận.

Ví dụ, trong không gian hai chiều, xét hai đối tượng $p = (1;2)$ và $q = (4;6)$:

$$d(p,q) = \sqrt{(1 - 4)^2 + (2 - 6)^2} = 5. \tag{2.3}$$

Nếu $\varepsilon = 5$ thì $q \in N_\varepsilon(p)$. Ngược lại, nếu $\varepsilon = 4$ thì q không thuộc lân cận ε của p. Như vậy, việc thay đổi ε có thể làm thay đổi số lượng đối tượng lân cận ngay cả khi dữ liệu không thay đổi.

Vì khoảng cách từ một điểm đến chính nó bằng 0, điểm đó cũng được tính trong lân cận của mình. Chương trình và các ví dụ của nhóm đều dùng quy ước này (Ester et al., 1996).

Ngưỡng mật độ MinPts

MinPts là số lượng đối tượng tối thiểu phải có trong lân cận ε để đối tượng đang xét được xác định là đối tượng lõi (Core).

Điều kiện xác định đối tượng lõi được biểu diễn như sau:

$$|N_\varepsilon(p)| \geq \text{MinPts}. \tag{2.4}$$

Trong đó, $|N_\varepsilon(p)|$ là số phần tử thuộc lân cận ε của p, bao gồm cả chính p. Điều kiện trên cho biết một đối tượng được xem là lõi khi số lượng đối tượng trong lân cận của nó đạt ít nhất ngưỡng MinPts.

MinPts là ngưỡng để xác định một đối tượng có đủ điều kiện mật độ để trở thành lõi và mở rộng cụm hay không, chứ không phải là số lượng thành viên cố định của mỗi cụm. Một cụm hợp lệ phải chứa ít nhất MinPts đối tượng, nhưng có thể tiếp tục mở rộng qua nhiều đối tượng lõi liên tiếp và có số lượng thành viên lớn hơn ngưỡng này.

Ảnh hưởng của hai tham số

ε (Eps) và MinPts quyết định mức mật độ tối thiểu mà DBSCAN sử dụng để nhận diện cụm. Nếu giữ nguyên MinPts và giảm ε, số đối tượng trong mỗi vùng lân cận thường giảm, khiến nhiều đối tượng không còn đủ điều kiện lõi. Ngược lại, tăng ε có thể làm xuất hiện thêm các kết nối giữa những đối tượng trước đó nằm ngoài lân cận của nhau.

Tương tự, khi giữ nguyên ε, việc tăng MinPts làm điều kiện trở thành lõi chặt chẽ hơn, còn giảm MinPts khiến các đối tượng dễ thỏa mãn điều kiện lõi hơn. Vì vậy, hai tham số cần được xem xét cùng lúc vì thay đổi về mật độ có thể làm thay đổi cả số lượng cụm, thành viên cụm và số đối tượng được xác định là nhiễu (Ester et al., 1996).

2.2.2 Đối tượng lõi, đối tượng biên và nhiễu

Dựa trên số lượng đối tượng trong lân cận ε và quan hệ với những đối tượng lõi, DBSCAN phân biệt ba loại đối tượng: lõi (Core), biên (Border) và nhiễu (Noise).

Đối tượng lõi (Core)

Đối tượng p được xác định là lõi nếu:

$$|N_\varepsilon(p)| \geq \text{MinPts}. \tag{2.5}$$

Đối tượng lõi có vai trò mở rộng cụm. Khi xét một đối tượng lõi, thuật toán có thể bổ sung các đối tượng nằm trong lân cận ε của nó vào cụm đang hình thành. Nếu trong số đó có đối tượng lõi khác, thuật toán tiếp tục xét lân cận của đối tượng lõi mới để mở rộng cụm.

Đối tượng biên (Border)

Đối tượng p được xác định là biên khi nó không thỏa mãn điều kiện lõi, nhưng nằm trong lân cận ε của ít nhất một đối tượng lõi q:

$$\begin{aligned} &|N_\varepsilon(p)| < \text{MinPts}, \\ \exists q \in D: \quad &|N_\varepsilon(q)| \geq \text{MinPts} \quad \wedge \quad p \in N_\varepsilon(q). \end{aligned} \tag{2.6}$$

Điều kiện thứ nhất cho biết đối tượng p không có đủ số lượng đối tượng trong lân cận để trở thành lõi. Điều kiện thứ hai yêu cầu tồn tại ít nhất một đối tượng q trong tập dữ liệu D, sao cho q có số lượng đối tượng trong lân cận lớn hơn hoặc bằng MinPts, tức là q là đối tượng lõi, đồng thời p nằm trong lân cận ε của q. Khi thỏa mãn cả hai điều kiện, p được xác định là đối tượng biên.

Vì không đáp ứng điều kiện lõi, đối tượng biên được bổ sung vào cụm nhưng không được sử dụng để tiếp tục mở rộng cụm sang những đối tượng khác.

Trong một số trường hợp, một đối tượng biên có thể nằm trong lân cận của các đối tượng lõi thuộc những cụm khác nhau. Khi đó, đối tượng biên vẫn chỉ được gán vào một cụm, nhưng cụm được chọn có thể phụ thuộc vào thứ tự duyệt hoặc quy tắc gán nhãn của thuật toán.

Nhiễu (Noise)

Đối tượng được xác định là nhiễu nếu không phải lõi và cũng không nằm trong lân cận ε của bất kỳ đối tượng lõi nào. Khi đó, đối tượng không thuộc cụm nào theo các tham số đang sử dụng.

Trạng thái nhiễu trong kết quả cuối cùng khác với việc một đối tượng được đánh dấu nhiễu trong quá trình duyệt. Nếu thuật toán xét một đối tượng chưa đủ điều kiện lõi trước khi phát hiện cụm chứa nó, đối tượng đó có thể tạm thời được đánh dấu nhiễu. Sau đó, nếu một đối tượng lõi khác có lân cận chứa đối tượng này, nó vẫn được bổ sung vào cụm dưới dạng đối tượng biên.

Như vậy, việc một đối tượng không thỏa mãn điều kiện lõi chưa đủ để kết luận đối tượng đó là nhiễu. Thuật toán còn phải xét khả năng đối tượng được kết nối với một lõi khác (Ester et al., 1996).

2.2.3 Nguyên lý mở rộng cụm

Để hình thành một cụm hoàn chỉnh, DBSCAN cần xác định quan hệ giữa những đối tượng thông qua ba khái niệm: khả đạt trực tiếp theo mật độ, khả đạt theo mật độ và kết nối theo mật độ.

Khả đạt trực tiếp theo mật độ (Directly Density-Reachable)

Đối tượng q được gọi là khả đạt trực tiếp theo mật độ từ đối tượng p nếu q nằm trong lân cận ε của p và p là đối tượng lõi:

$$\begin{cases} q \in N_\varepsilon(p), \\ |N_\varepsilon(p)| \geq \text{MinPts}. \end{cases} \tag{2.7}$$

Quan hệ khả đạt trực tiếp theo mật độ không có tính đối xứng trong mọi trường hợp. Ví dụ, nếu p là lõi và q là biên thì q có thể khả đạt trực tiếp từ p. Tuy nhiên, p không khả đạt trực tiếp từ q vì q không đáp ứng điều kiện lõi. Ngược lại, nếu cả hai đối tượng đều là lõi và cách nhau không vượt quá ε thì quan hệ khả đạt trực tiếp tồn tại theo cả hai chiều.

Khả đạt theo mật độ (Density-Reachable)

Một đối tượng q được gọi là khả đạt theo mật độ từ đối tượng p nếu có thể đi từ p đến q thông qua một chuỗi các đối tượng. Trong chuỗi này, mỗi đối tượng tiếp theo phải khả đạt trực tiếp theo mật độ từ đối tượng đứng ngay trước nó.

Ví dụ, xét ba đối tượng p, r và q, trong đó p và r đều là lõi. Nếu r nằm trong lân

cận ε của p, đồng thời q nằm trong lân cận ε của r, thì q khả đạt theo mật độ từ p. Khi đó, p và q không nhất thiết phải nằm trong lân cận ε của nhau.

Điểm cuối cùng có thể là lõi hoặc biên nhưng điểm biên không thể làm điểm trung gian để mở rộng chuỗi.

Kết nối theo mật độ (Density-Connected)

Hai đối tượng p và q được gọi là kết nối theo mật độ nếu tồn tại một đối tượng o sao cho cả p và q đều khả đạt theo mật độ từ o.

Hai đối tượng biên có thể không khả đạt theo mật độ từ nhau nhưng vẫn thuộc cùng một cụm nếu chúng đều khả đạt theo mật độ từ một đối tượng lõi chung. Do đó, các đối tượng trong cùng một cụm không nhất thiết phải nằm trong lân cận ε của nhau mà có thể được liên kết thông qua các chuỗi đối tượng lõi trung gian.

Điều kiện hình thành cụm

Dựa trên các quan hệ trên, một cụm trong DBSCAN được xác định bởi hai điều kiện:

- **Tính kết nối (Connectivity):** Mọi cặp đối tượng thuộc cùng một cụm phải kết nối theo mật độ với nhau.
- **Tính tối đa (Maximality):** Nếu một đối tượng thuộc cụm thì mọi đối tượng khả đạt theo mật độ từ đối tượng đó cũng phải thuộc cụm.

Hai điều kiện này bảo đảm rằng mỗi cụm bao gồm các đối tượng liên kết với nhau theo mật độ và được mở rộng tối đa theo quan hệ khả đạt (Ester et al., 1996).

Nói gọn lại, DBSCAN bắt đầu từ một điểm lõi, đi qua các điểm lõi kề nhau, rồi đưa các điểm biên gần những lõi này vào cụm. Khi không còn lõi nào mở rộng được, cụm đã tìm xong. Từ đó có thể giải thích được vì sao cụm có thể dài hoặc cong: hai đầu cụm không cần nằm cách nhau dưới ε, miễn là còn một đường nối qua các điểm lõi.

Khi dữ liệu thay đổi, Incremental DBSCAN dựa vào các lõi và đường nối đã lưu để kiểm tra phần cụm bị ảnh hưởng, thay vì bắt đầu lại từ đầu (Ester et al., 1998).

2.2.4 Quy trình thực hiện

Trong chương trình của nhóm, lớp `DBSCAN` dùng phương thức `fit()` để phân cụm ban đầu. Các bước chính là:

Bước 1. Khởi tạo dữ liệu và thông tin lân cận.

Mỗi đối tượng được gán một mã định danh, trạng thái chưa phân loại và nhãn

cụm chưa xác định. Thuật toán truy vấn lân cận ε của từng đối tượng và lưu các đối tượng lân cận để sử dụng trong quá trình mở rộng cụm.

Bước 2. Xét một đối tượng chưa được phân loại.

Nếu số phần tử trong lân cận ε nhỏ hơn MinPts, đối tượng được đánh dấu nhiễu tại thời điểm xét. Ngược lại, nếu đáp ứng điều kiện lõi, thuật toán tạo một cụm mới và bắt đầu mở rộng từ đối tượng đó.

Bước 3. Mở rộng cụm qua các đối tượng lõi.

Những đối tượng trong lân cận của lõi ban đầu được đưa vào ngăn xếp (stack). Ngăn xếp xử lý theo nguyên tắc LIFO: đối tượng được đưa vào sau sẽ được lấy ra trước. Mỗi lượt, thuật toán lấy đối tượng ở đỉnh ngăn xếp và xác định loại dựa trên số lượng lân cận. Nếu là lõi, các đối tượng trong lân cận của nó được đưa tiếp vào ngăn xếp. Nếu là biên, đối tượng được bổ sung vào cụm nhưng không làm cụm mở rộng thêm.

Bước 4. Hoàn tất cụm hiện tại.

Khi ngăn xếp không còn đối tượng cần xử lý, thuật toán kết thúc cụm hiện tại. Sau đó, thuật toán tiếp tục xét những đối tượng chưa được phân loại để tìm các cụm khác.

Bước 5. Trả về kết quả phân cụm.

Quá trình kết thúc khi tất cả đối tượng đã được xử lý. Mỗi đối tượng có một nhãn cụm tương ứng, những đối tượng không thuộc cụm nào được gán nhãn nhiễu.

Quy trình trên trình bày cách nhóm hiện thực DBSCAN; các khái niệm lân cận, lõi, mở rộng cụm và nhiễu dựa trên thuật toán gốc. Cách dùng stack LIFO để mở rộng cụm được mô tả trong paper Incremental DBSCAN (Ester et al., 1996, 1998).

Trong cách cài đặt hiện tại, các đối tượng lân cận được lưu trong thuộc tính `neighbor_ids` của lớp `DataPoint`. Phương thức `_region_query()` xác định những đối tượng có khoảng cách không vượt quá ε, còn `_expand_cluster()` sử dụng stack LIFO để mở rộng cụm qua các đối tượng lõi. Mã sắp xếp mã định danh lân cận trước khi đưa vào stack để thứ tự duyệt ổn định.

Việc lưu thông tin lân cận không chỉ phục vụ lần phân cụm ban đầu mà còn hỗ trợ các thao tác cập nhật tăng dần được trình bày ở phần Incremental DBSCAN.

2.2.5 Ví dụ minh họa DBSCAN từng bước

Xét tập dữ liệu gồm 13 đối tượng trong không gian hai chiều. Sử dụng khoảng cách Euclid và xác định các tham số:

$$\varepsilon = 1{,}5, \qquad \text{MinPts} = 4. \tag{2.8}$$

Tọa độ và kết quả xác định lân cận của từng đối tượng được trình bày trong Bảng 2.1.

Bảng 2.1: Lân cận ε và phân loại đối tượng trong ví dụ DBSCAN

Điểm	**Tọa độ**	$N_\varepsilon(p)$	$\lvert N_\varepsilon(p)\rvert$	**Loại**
P_1	$(1;1)$	$\{P_1,P_2,P_3,P_4\}$	4	Lõi
P_2	$(2;1)$	$\{P_1,P_2,P_3,P_4,P_5\}$	5	Lõi
P_3	$(1;2)$	$\{P_1,P_2,P_3,P_4\}$	4	Lõi
P_4	$(2;2)$	$\{P_1,P_2,P_3,P_4,P_5\}$	5	Lõi
P_5	$(3;2)$	$\{P_2,P_4,P_5,P_6\}$	4	Lõi
P_6	$(4;3)$	$\{P_5,P_6\}$	2	Biên
P_7	$(7;7)$	$\{P_7,P_8,P_9,P_{10}\}$	4	Lõi
P_8	$(8;7)$	$\{P_7,P_8,P_9,P_{10},P_{11}\}$	5	Lõi
P_9	$(7;8)$	$\{P_7,P_8,P_9,P_{10}\}$	4	Lõi
P_{10}	$(8;8)$	$\{P_7,P_8,P_9,P_{10},P_{11}\}$	5	Lõi
P_{11}	$(9;8)$	$\{P_8,P_{10},P_{11}\}$	3	Biên
P_{12}	$(5;5)$	$\{P_{12}\}$	1	Nhiễu
P_{13}	$(11;3)$	$\{P_{13}\}$	1	Nhiễu

Bước 1: Xác định lân cận và phân loại đối tượng.

Với $\varepsilon = 1{,}5$, hai đối tượng được xem là lân cận nếu khoảng cách Euclid giữa chúng không vượt quá 1,5.

Ví dụ, đối tượng $P_1 = (1;1)$ có khoảng cách đến $P_4 = (2;2)$ là:

$$d(P_1,P_4) = \sqrt{(1\text{ - }2)^2 + (1\text{ - }2)^2} = \sqrt{2} \approx 1{,}41 < 1{,}5. \tag{2.9}$$

Do đó, $P_4 \in N_\varepsilon(P_1)$. Tương tự, P_2 và P_3 cũng thuộc lân cận của P_1. Tính cả chính P_1, ta có:

$$N_\varepsilon(P_1) = \{P_1, P_2, P_3, P_4\}. \tag{2.10}$$

Vì $|N_\varepsilon(P_1)| = 4 = \text{MinPts}$ nên P_1 là lõi.

P_6 chỉ có hai đối tượng trong lân cận là P_5 và chính nó. Mặc dù không đáp ứng điều kiện lõi, P_6 vẫn nằm trong lân cận của P_5, một đối tượng lõi, nên được xác định là biên.

Bước 2: Mở rộng cụm thứ nhất.

Giả sử thuật toán bắt đầu từ P_1. Vì P_1 là lõi nên một cụm mới C_1 được tạo ra với các đối tượng trong $N_\varepsilon(P_1)$.

Khi xét P_2, thuật toán xác định P_2 cũng là lõi. Lân cận của P_2 chứa thêm P_5, vì vậy P_5 được bổ sung vào quá trình mở rộng cụm.

Tiếp tục xét P_5, ta có:

$$N_\varepsilon(P_5) = \{P_2, P_4, P_5, P_6\}. \tag{2.11}$$

Vì P_5 là lõi nên P_6 được bổ sung vào C_1. P_6 không đủ điều kiện lõi nên quá trình mở rộng không tiếp tục từ đối tượng này.

Sau khi xử lý các đối tượng liên quan, cụm thứ nhất là:

$$C_1 = \{P_1, P_2, P_3, P_4, P_5, P_6\}. \tag{2.12}$$

Khoảng cách giữa P_1 và P_6 lớn hơn ε, nhưng P_6 vẫn thuộc C_1 thông qua chuỗi khả đạt theo mật độ từ các đối tượng lõi trung gian. Điều này nói lên rằng các thành viên trong cùng một cụm không nhất thiết phải nằm trong lân cận trực tiếp của nhau.

Bước 3: Mở rộng cụm thứ hai.

Sau khi hoàn tất C_1, thuật toán tiếp tục xét P_7, một đối tượng lõi chưa được phân loại. Từ P_7, các đối tượng P_8, P_9 và P_{10} được bổ sung vào cụm mới C_2.

Trong quá trình mở rộng, P_8 và P_{10} đều là lõi và có P_{11} trong lân cận ε. Vì vậy, P_{11} được gán vào C_2 dưới dạng đối tượng biên.

Kết quả cụm thứ hai là:

$$C_2 = \{P_7, P_8, P_9, P_{10}, P_{11}\}. \tag{2.13}$$

Bước 4: Xác định nhiễu và kết quả cuối cùng.

Hai đối tượng P_{12} và P_{13} chỉ có chính chúng trong lân cận ε, đồng thời không nằm trong lân cận của bất kỳ đối tượng lõi nào. Vì vậy, cả hai được xác định là nhiễu.

Kết quả phân cụm cuối cùng gồm hai cụm C_1, C_2 và tập nhiễu:

$$N = \{P_{12}, P_{13}\}. \tag{2.14}$$

Ví dụ trên cho thấy DBSCAN không yêu cầu mọi đối tượng trong cùng một cụm phải nằm gần nhau trong phạm vi ε. Thay vào đó, một cụm được hình thành từ những đối tượng có quan hệ kết nối theo mật độ, trong đó các đối tượng lõi quyết định khả năng mở rộng cụm (Ester et al., 1996).

2.2.6 Ưu điểm và hạn chế

Ưu điểm

DBSCAN có khả năng xác định số lượng cụm từ cấu trúc phân bố của dữ liệu mà không yêu cầu khai báo trước số cụm k. Ngoài ra, do cụm được hình thành thông qua quan hệ kết nối theo mật độ, thuật toán có thể nhận diện những cụm có hình dạng không lồi, chẳng hạn cụm phân bố theo đường cong hoặc vòng cung.

Một ưu điểm khác là khả năng nhận diện nhiễu. Những đối tượng không thỏa mãn điều kiện thuộc bất kỳ cụm nào không bị bắt buộc gán vào cụm gần nhất mà được phân loại riêng. Điều này giúp phân biệt các vùng có đủ mật độ với những đối tượng nằm tách biệt khỏi cấu trúc cụm (Ester et al., 1996).

Hạn chế

Hạn chế của DBSCAN là kết quả phụ thuộc vào ε (Eps) và MinPts. Nếu ε quá nhỏ, nhiều đối tượng có thể không đủ điều kiện lõi và bị phân loại thành nhiễu. Ngược lại, ε quá lớn có thể tạo thêm các kết nối giữa những vùng vốn được xem là những cụm riêng biệt. Tương tự, thay đổi MinPts cũng ảnh hưởng đến khả năng hình thành và mở rộng cụm.

Việc sử dụng một bộ tham số chung cho toàn bộ tập dữ liệu cũng làm DBSCAN gặp khó khăn khi các cụm có mật độ chênh lệch lớn. Một giá trị ε phù hợp với cụm có mật độ cao chưa chắc phù hợp với cụm có mật độ thấp. Trong trường hợp đó, một số cụm có thể bị chia nhỏ, bị gộp hoặc có nhiều thành viên bị xác định là nhiễu (Ester et al., 1996).

Ngoài ra, một đối tượng biên có thể nằm trong lân cận của các đối tượng lõi thuộc

hai cụm khác nhau. Khi đó, đối tượng biên có thể được gán vào một trong các cụm liên quan tùy theo thứ tự xử lý, nhưng bản thân đối tượng biên không tạo ra kết nối để hợp nhất hai cụm. Vì vậy, các cách xử lý khác nhau có thể dẫn đến sự khác biệt trong việc gán cụm cho một số đối tượng biên, dù cấu trúc kết nối giữa các đối tượng lõi không thay đổi.

Trong trường hợp dữ liệu thay đổi thường xuyên, DBSCAN thông thường không cập nhật kết quả sau mỗi phép chèn hoặc xóa. Nếu thực hiện lại toàn bộ quá trình phân cụm sau mỗi thay đổi, những quan hệ lân cận và kết nối cụm không bị tác động vẫn phải được xử lý lại (Ester et al., 1998).

2.3. Thuật toán Incremental DBSCAN

Incremental DBSCAN mở rộng DBSCAN để cập nhật kết quả khi thêm hoặc xóa điểm. Thay vì phân cụm lại từ đầu sau mỗi thay đổi, thuật toán dùng thông tin đã lưu để tìm những điểm và kết nối cần cập nhật. Phần này dựa trên bài báo gốc về Incremental DBSCAN năm 1998, *Incremental Clustering for Mining in a Data Warehousing Environment* (Ester et al., 1998).

2.3.1 Nhu cầu cập nhật cụm khi dữ liệu thay đổi

Kết quả của DBSCAN phụ thuộc vào số đối tượng trong từng lân cận ε và quan hệ kết nối giữa các đối tượng lõi. Do đó, việc thêm hoặc xóa một đối tượng có thể làm thay đổi kết quả phân cụm ngay cả khi hai tham số ε và MinPts được giữ nguyên.

Khi chèn một đối tượng, số phần tử trong lân cận của các đối tượng gần nó có thể tăng lên. Một đối tượng trước đó là biên hoặc nhiễu có thể trở thành lõi, từ đó tạo thêm kết nối, hình thành cụm mới hoặc hợp nhất các cụm cũ. Ngược lại, việc xóa một đối tượng có thể khiến một số lõi không còn đạt ngưỡng MinPts. Tùy thuộc vào cấu trúc cụm và những kết nối mật độ còn lại, phép xóa có thể làm thu hẹp cụm, khiến cụm biến mất hoặc dẫn đến tách cụm nếu các phần còn lại không còn kết nối theo mật độ.

Có thể xử lý các thay đổi này bằng cách chạy lại DBSCAN trên toàn bộ dữ liệu sau mỗi lần cập nhật. Tuy nhiên, cách làm đó phải thực hiện lại nhiều phép kiểm tra đối với những đối tượng không liên quan đến vị trí thêm hoặc xóa. Incremental DBSCAN tận dụng thông tin đã có về lân cận, tính chất lõi và thành viên cụm để hạn chế việc tính toán lặp lại.

Khi thêm hoặc xóa một đối tượng, chỉ những đối tượng nằm trong lân cận ε của nó mới có thể thay đổi số lượng đối tượng lân cận và tính chất lõi. Tuy nhiên, sự thay đổi

này có thể khiến hai cụm hợp nhất hoặc một cụm bị tách ra. Khi đó, nhãn cụm của nhiều đối tượng khác cũng có thể thay đổi, kể cả những đối tượng nằm ngoài phạm vi ε của điểm được cập nhật. Vì vậy, thuật toán cần xác định cả những đối tượng có thể thay đổi tính chất lõi và những cụm có thể bị ảnh hưởng bởi phép cập nhật (Ester et al., 1998).

Trong đồ án, nhóm xét từng phép chèn (INSERT) hoặc xóa (DELETE) một đối tượng, đồng thời giữ cố định hàm khoảng cách và hai tham số ε, MinPts trong suốt quá trình cập nhật để xác định những thay đổi về tính chất lõi và cấu trúc cụm do chính đối tượng được thêm hoặc xóa gây ra, thay vì do sự thay đổi tham số.

2.3.2 Vùng bị ảnh hưởng và tập điểm cần cập nhật

Phạm vi thay đổi sau một phép cập nhật

Xét đối tượng p được chèn vào hoặc xóa khỏi tập dữ liệu. Để phân biệt hai trạng thái, ký hiệu D^- là tập dữ liệu không chứa p và $D^+ = D^- \cup \{p\}$ là tập dữ liệu có chứa p.

Như vậy, phép chèn làm tập dữ liệu chuyển từ D^- sang D^+, còn phép xóa thực hiện theo chiều ngược lại.

Vì điều kiện lõi phụ thuộc vào số phần tử trong lân cận ε, những đối tượng có thể thay đổi tính chất lõi phải nằm trong $N_\varepsilon(p)$. Đối với các đối tượng nằm ngoài vùng này, số lượng phần tử trong lân cận của chúng không thay đổi khi chèn hoặc xóa p.

Sự thay đổi tính chất lõi của một đối tượng gần vị trí cập nhật còn có thể ảnh hưởng đến trạng thái biên hoặc nhiễu của những đối tượng xung quanh nó. Ví dụ, khi đối tượng q nằm trong lân cận ε của p chuyển thành lõi, những đối tượng nằm trong lân cận của q có thể chuyển từ nhiễu thành biên.

Do q nằm cách p không quá ε, các đối tượng nằm trong lân cận của q sẽ cách p không quá 2ε. Vì vậy, phạm vi có thể thay đổi trạng thái biên và nhiễu được xét đến 2ε tính từ vị trí cập nhật.

Từ đó, phạm vi ảnh hưởng cục bộ được phân biệt thành ba vùng:

Bảng 2.2: Ảnh hưởng cục bộ của phép chèn hoặc xóa đối tượng

Vùng	**Khả năng thay đổi**
$N_\varepsilon(p)$	Số lượng đối tượng lân cận thay đổi, một số đối tượng có thể chuyển giữa trạng thái lõi và không lõi.
$N_{2\varepsilon}(p) \setminus N_\varepsilon(p)$ (vùng nằm giữa hai bán kính ε và 2ε)	Tính chất lõi không thay đổi, nhưng đối tượng không lõi có thể chuyển giữa biên và nhiễu.
Ngoài $N_{2\varepsilon}(p)$	Tính chất lõi và trạng thái biên hoặc nhiễu không thay đổi trực tiếp do phép cập nhật. Tuy nhiên, nhãn cụm vẫn có thể thay đổi nếu các cụm liên quan bị hợp nhất hoặc tách.

Cần phân biệt thay đổi tính chất lõi với thay đổi thành viên cụm. Giới hạn 2ε chỉ mô tả phạm vi mà trạng thái lõi, biên và nhiễu có thể thay đổi trực tiếp. Khi một kết nối giữa các đối tượng lõi được tạo thêm hoặc mất đi, tác động đến cấu trúc cụm có thể lan xa hơn phạm vi này (Ester et al., 1998).

Chẳng hạn, nếu một đối tượng mới tạo ra kết nối giữa hai cụm, tất cả thành viên của hai cụm sẽ được xem là thuộc cùng một cụm sau cập nhật, kể cả những đối tượng ở xa vị trí chèn.

Tập đối tượng bị ảnh hưởng

Tập đối tượng bị ảnh hưởng (Affected Objects) gồm những điểm có thể đổi trạng thái hoặc nhãn cụm sau khi một điểm p được thêm hoặc xóa.

Để tìm tập này, thuật toán xét các điểm trong lân cận ε của điểm được thêm hoặc xóa, rồi lần theo các kết nối theo mật độ từ những điểm đó. Vì vậy, tập có thể gồm cả những điểm cách điểm cập nhật xa hơn một bán kính ε. Không phải điểm nào trong tập cũng chắc chắn đổi cụm; đây là vùng cần kiểm tra.

Đối với thao tác INSERT, các kết nối mật độ được xét trên tập dữ liệu sau khi thêm p, nhằm xác định những kết nối mới có thể được hình thành. Ngược lại, đối với DELETE, các kết nối được xét trên tập dữ liệu trước khi xóa p, nhằm xác định những kết nối có khả năng bị mất sau khi xóa.

Việc xác định tập Affected Objects giúp nhóm tập trung kiểm tra phần dữ liệu có liên quan thay vì tính lại mọi thứ. Một số điểm trong tập có thể giữ nguyên nhãn sau khi kiểm tra.

Xác định những đối tượng thay đổi tính chất lõi

Để xác định các đối tượng thay đổi tính chất lõi khi thêm hoặc xóa điểm p, thuật toán chỉ cần kiểm tra những đối tượng nằm trong lân cận ε của p, thay vì tính toán lại số lượng lân cận của toàn bộ dữ liệu.

Đối với thao tác INSERT, việc thêm p làm tăng số lượng đối tượng lân cận của mỗi điểm nằm trong phạm vi ε của p thêm một đơn vị. Vì vậy, những đối tượng trước đó có đúng MinPts - 1 phần tử trong lân cận sẽ trở thành đối tượng lõi sau khi thêm p. Bản thân điểm p cũng được kiểm tra để xác định xem nó có đủ điều kiện trở thành lõi hay không.

Ngược lại, đối với thao tác DELETE, việc xóa p làm giảm số lượng đối tượng lân cận của các điểm nằm trong phạm vi ε của p đi một đơn vị. Do đó, những đối tượng trước đó có đúng MinPts phần tử trong lân cận sẽ mất tính chất lõi sau khi xóa p.

Nhờ lưu trữ số lượng đối tượng lân cận từ kết quả phân cụm trước đó, thuật toán có thể xác định những thay đổi về tính chất lõi mà không cần tính toán lại trên toàn bộ tập dữ liệu.

Tập khởi đầu cập nhật UpdSeed

Affected Objects xác định phạm vi có thể thay đổi, nhưng việc duyệt lại toàn bộ tập này vẫn có thể tốn nhiều chi phí. Incremental DBSCAN sử dụng tập đối tượng khởi đầu cập nhật (Update Seeds), ký hiệu UpdSeed, để tập trung kiểm tra các quan hệ kết nối có thể thay đổi.

Đối với phép chèn, gọi NewCore là tập các đối tượng trở thành lõi sau khi thêm p, bao gồm chính p nếu đối tượng mới thỏa mãn điều kiện lõi:

$$\text{NewCore} = \left\{ q \in D^{+} \;\middle|\; \begin{array}{l} q \text{ là lõi trong } D^{+}, \\ q \text{ không là lõi trong } D^{-} \end{array} \right\}. \tag{2.15}$$

Tập khởi đầu cập nhật khi chèn được xác định:

$$\text{UpdSeed}_{Ins} = \left\{ q \in D^{+} \;\middle|\; \begin{array}{l} q \text{ là lõi trong } D^{+}, \\ \exists r \in \text{NewCore} : \; q \in N_{\varepsilon}^{D^{+}}(r) \end{array} \right\}. \tag{2.16}$$

Như vậy, UpdSeed_{Ins} gồm những đối tượng lõi nằm trong lân cận của ít nhất một đối tượng vừa trở thành lõi. Các đối tượng này được sử dụng để xác định xem những kết nối mới có tạo cụm, mở rộng cụm hay nối các cụm cũ với nhau hay không.

Đối với phép xóa, gọi LostCore là tập những đối tượng từng là lõi trong D^{+} nhưng

không còn là lõi trong D^-. Tập này bao gồm p nếu đối tượng bị xóa vốn là lõi.

Tập khởi đầu cập nhật khi xóa được xác định:

$$\text{UpdSeed}_{Del} = \left\{ q \in D^- \;\middle|\; \begin{array}{l} q \text{ là lõi trong } D^-, \\ \exists r \in \text{LostCore} : \; q \in N_\varepsilon^{D^+}(r) \end{array} \right\}. \tag{2.17}$$

Tập này gồm những đối tượng vẫn là lõi sau phép xóa và nằm gần các đối tượng vừa mất tính chất lõi. Nếu các đối tượng thuộc UpdSeed_{Del} không còn kết nối trực tiếp với nhau, thuật toán cần kiểm tra xem có tồn tại chuỗi lõi khác duy trì kết nối hay không (Ester et al., 1998).

2.3.3 Các trường hợp khi thêm điểm

Nguyên tắc cập nhật khi chèn

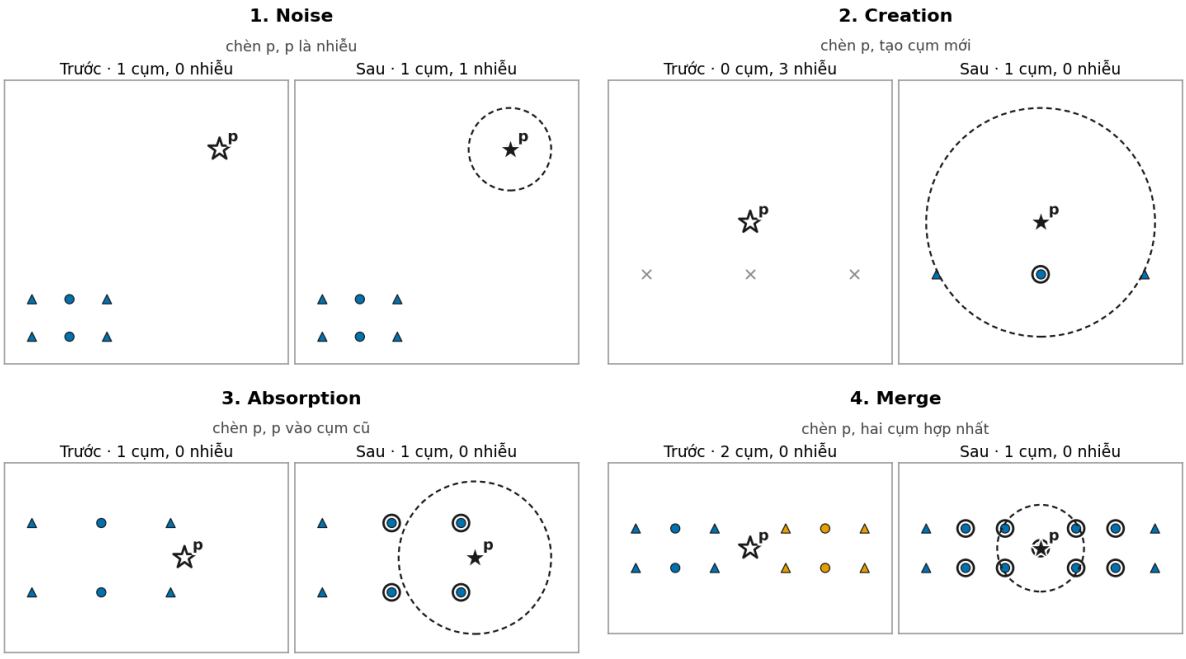
Giả sử tập D^- đã được phân cụm và một đối tượng mới p được chèn vào, tạo thành tập $D^+ = D^- \cup \{p\}$.

Phép chèn chỉ làm tăng hoặc giữ nguyên số lượng đối tượng trong các lân cận. Vì vậy, những đối tượng cũ đã là lõi vẫn tiếp tục là lõi, trong khi một số đối tượng không lõi có thể trở thành lõi. Các kết nối theo mật độ có thể được tạo thêm nhưng không bị mất đi.

Về kết quả phân cụm, có thể xảy ra bốn trường hợp: Noise, Creation, Absorption và Merge (Ester et al., 1998).

Bảng 2.3: Các trường hợp cập nhật khi chèn đối tượng

Trường hợp	**Điều kiện nhận biết**	**Kết quả**
Noise	Không có lõi mới và p không nằm trong lân cận của lõi cũ.	p là nhiễu, các cụm cũ giữ nguyên.
Creation	Các kết nối do lõi mới tạo ra không liên hệ với cụm đã tồn tại.	Hình thành một cụm mới.
Absorption	Đối tượng mới nằm trong lân cận của ít nhất một lõi thuộc cụm cũ mà không tạo ra lõi mới, hoặc các kết nối do lõi mới hình thành chỉ liên hệ với một cụm cũ.	Đối tượng mới và có thể một số đối tượng nhiễu được bổ sung vào một cụm hiện có, không làm hợp nhất các cụm cũ.
Merge	Các kết nối mới liên hệ những lõi thuộc ít nhất hai cụm cũ.	Các cụm liên quan được hợp nhất.



Hình 2.1: Minh họa các trường hợp cập nhật khi chèn đối tượng: Noise, Creation, Absorption và Merge

Trong trường hợp Noise, đối tượng mới không đủ điều kiện trở thành lõi và không nằm trong lân cận của bất kỳ đối tượng lõi nào, do đó được xác định là nhiễu và không làm thay đổi các cụm hiện có.

Trong trường hợp Creation, phép chèn khiến một hoặc nhiều đối tượng trở thành

lõi, hình thành các kết nối theo mật độ giữa những đối tượng trước đó chưa thuộc cụm nào. Kết quả là một cụm mới được tạo thành mà không ảnh hưởng đến các cụm đã tồn tại.

Trong trường hợp Absorption, đối tượng mới được bổ sung vào một cụm hiện có mà không làm hợp nhất các cụm khác nhau. Điều này có thể xảy ra khi đối tượng mới trở thành biên của một cụm đã tồn tại, hoặc khi các đối tượng lõi mới chỉ thiết lập thêm kết nối với một cụm cũ.

Để tránh lặp lại cùng một quy trình ba lần, ví dụ chi tiết dưới đây tập trung vào Merge, trường hợp cần theo dõi rõ các điểm mới thành lõi và hai cụm cũ được nối với nhau.

Trường hợp 4 - Merge (Hợp nhất cụm)

Merge xảy ra khi các kết nối thông qua lõi mới liên hệ những đối tượng lõi thuộc ít nhất hai cụm khác nhau trước cập nhật. Khi đó, các cụm liên quan trở thành một cụm sau khi chèn.

Xét hai cụm ban đầu với $\varepsilon = 1{,}1$ và $\text{MinPts} = 4$.

Trong ví dụ này, tọa độ được ghi theo cặp $(x\,;y)$: thành phần thứ nhất là x, thành phần thứ hai là y. Dấu phẩy trong các giá trị như 2,8 là dấu thập phân; mỗi điểm chỉ có hai thành phần và không có tọa độ z.

Cụm thứ nhất gồm sáu đối tượng:

$$\begin{aligned} C_1 = \{&A_1 = (\text{-}1\,;0), A_2 = (0\,;0), A_3 = (1\,;0),\\ &A_4 = (\text{-}1\,;1), A_5 = (0\,;1), A_6 = (1\,;1)\}. \end{aligned} \tag{2.18}$$

Cụm thứ hai cũng gồm sáu đối tượng:

$$\begin{aligned} C_2 = \{&B_1 = (2{,}8\,;0), B_2 = (3{,}8\,;0), B_3 = (4{,}8\,;0),\\ &B_4 = (2{,}8\,;1), B_5 = (3{,}8\,;1), B_6 = (4{,}8\,;1)\}. \end{aligned} \tag{2.19}$$

Trước cập nhật, A_2, A_5, B_2 và B_5 là các đối tượng lõi. Hai cụm chưa có kết nối theo mật độ.

Chèn đối tượng $p = (1{,}9\,;0{,}5)$. Khoảng cách từ p đến các đối tượng A_3, A_6, B_1, B_4 đều bằng:

$$\sqrt{0{,}9^2 + 0{,}5^2} = \sqrt{1{,}06} \approx 1{,}03 < 1{,}1. \tag{2.20}$$

Vì vậy:

$$N_\varepsilon(p) = \{p, A_3, A_6, B_1, B_4\}. \tag{2.21}$$

Đối tượng p có năm phần tử trong lân cận nên trở thành lõi. Đồng thời, A_3, A_6, B_1 và B_4 đều tăng số lượng lân cận từ 3 lên 4, do đó cũng trở thành lõi.

Tập lõi mới gồm:

$$\text{NewCore} = \{p, A_3, A_6, B_1, B_4\}. \tag{2.22}$$

Khi xác định các đối tượng lõi trong lân cận của NewCore, ta thu được:

$$\begin{aligned}\text{UpdSeed}_{Ins} = \{&A_2, A_3, A_5, A_6,\\ &B_1, B_2, B_4, B_5, p\}.\end{aligned} \tag{2.23}$$

Tập này chứa các lõi thuộc cả C_1 và C_2. Đặc biệt, lõi p kết nối với những lõi mới ở hai phía, tạo ra đường kết nối mật độ giữa hai cụm.

Vì vậy, thuật toán hợp nhất C_1, C_2 và p thành một cụm gồm 13 đối tượng:

$$C = C_1 \cup C_2 \cup \{p\}. \tag{2.24}$$

Trong chương trình của nhóm, việc hợp nhất được thực hiện bằng cách giữ lại một nhãn cụm và cập nhật nhãn của các thành viên thuộc cụm còn lại. Thuật toán xác định những cụm cần hợp nhất dựa trên các kết nối mới giữa những đối tượng lõi, thay vì tính lại lân cận của toàn bộ 13 đối tượng. Tuy nhiên, chương trình vẫn cần duyệt các đối tượng đang lưu để cập nhật nhãn của những đối tượng thuộc cụm bị hợp nhất.

Quy trình INSERT trong chương trình của nhóm

Phương thức `insert()` được cài đặt dựa trên việc xác định những đối tượng vừa trở thành lõi và các cụm có kết nối với chúng. Quy trình được tóm tắt qua các bước sau:

Bước 1. Thêm p vào tập dữ liệu với ID duy nhất. Truy vấn $N_\varepsilon(p)$, lưu lân cận của p và bổ sung ID của p vào lân cận đã lưu của các đối tượng liên quan.

Bước 2. Xác định NewCore từ các đối tượng có số lượng lân cận vừa đạt ngưỡng MinPts, đồng thời xét điều kiện lõi của chính p.

Bước 3. Tạo UpdSeed_{Ins} từ các đối tượng lõi nằm trong lân cận của NewCore.

Bước 4. Nếu không có lõi mới, kiểm tra các lõi cũ trong lân cận của p. Nếu có lõi cũ, gán p thành biên của một cụm hợp lệ, ngược lại, gán p là nhiễu.

Bước 5. Nếu có lõi mới, dùng stack LIFO để mở rộng từ các lõi mới thuộc UpdSeed_{Ins}. Những lõi cũ trong tập seed cho biết các cụm đã tồn tại có kết nối với phần vừa cập nhật.

Bước 6. Dựa vào nhãn cụm cũ trong nhóm seed, tạo cụm mới khi chưa nối với cụm cũ, hấp thụ vào một cụm nếu chỉ nối với một cụm, hoặc hợp nhất các cụm nếu có kết nối qua lõi mới. Sau đó, cập nhật lõi mới và phân loại lại các đối tượng không lõi nằm gần chúng.

Phần mở rộng chỉ đi qua các lõi vừa phát sinh. Khi gặp lõi cũ trong UpdSeed_{Ins}, chương trình dùng nhãn cụm đã có để xác định cụm cần giữ hoặc hợp nhất, không phân cụm lại toàn bộ dữ liệu.

Một đối tượng biên có thể nằm trong lân cận của các đối tượng lõi thuộc nhiều cụm khác nhau, nhưng bản thân nó không có khả năng mở rộng quan hệ khả đạt theo mật độ. Vì vậy, sự xuất hiện của đối tượng biên này không làm hợp nhất các cụm. Việc hợp nhất chỉ xảy ra khi hình thành kết nối theo mật độ giữa các đối tượng lõi thuộc những cụm tách biệt trước đó.

2.3.4 Các trường hợp khi xóa điểm

Nguyên tắc cập nhật khi xóa

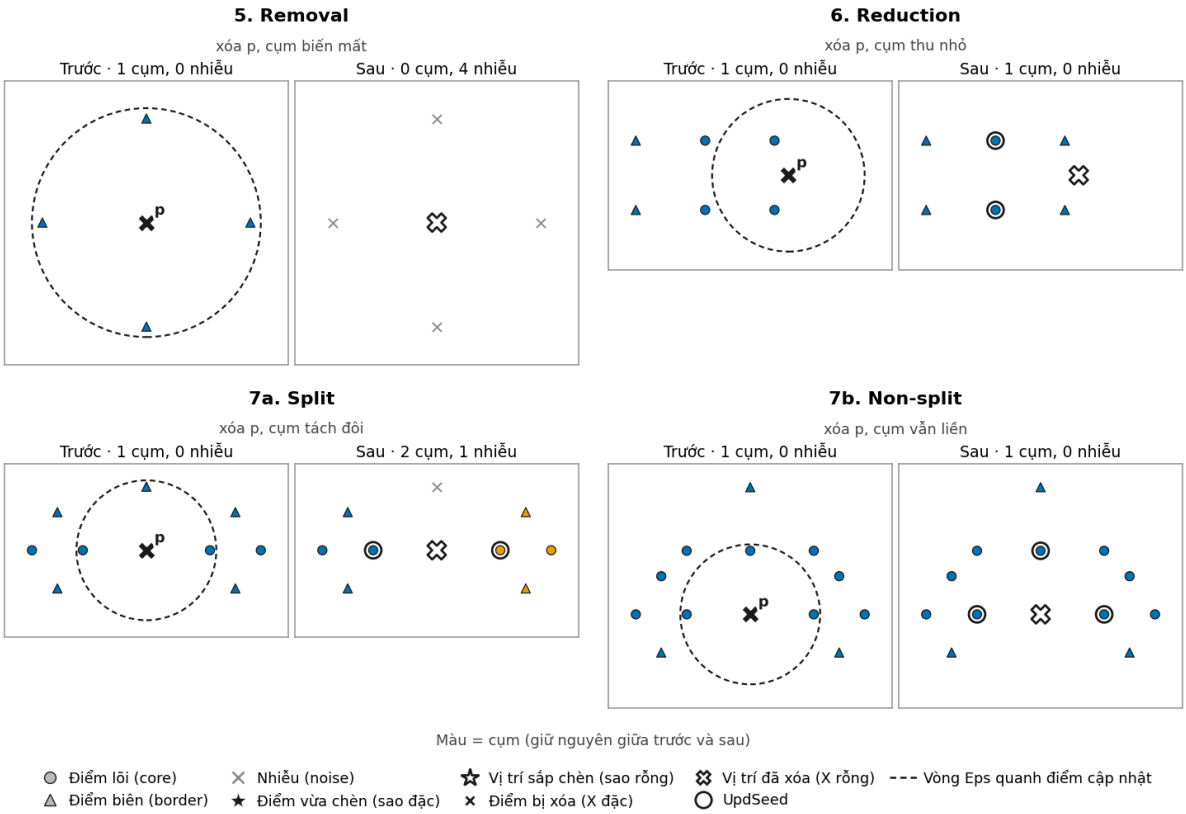
Khi xóa một đối tượng p, số lượng phần tử trong lân cận của những đối tượng còn lại chỉ có thể giảm hoặc giữ nguyên. Vì vậy, những đối tượng trước đó không phải lõi không thể trở thành lõi do phép xóa, ngược lại, một số đối tượng lõi có thể mất tính chất lõi.

Phép xóa không tạo thêm các kết nối theo mật độ. Tuy nhiên, việc mất một đối tượng lõi không nhất thiết khiến cụm bị tách, bởi các đối tượng còn lại vẫn có thể được liên kết thông qua những lõi khác.

Ba trường hợp chính khi xóa là Removal, Reduction và Potential Split. Riêng Potential Split cần phân biệt hai kết quả: Split và Non-split (Ester et al., 1998).

Bảng 2.4: Các trường hợp cập nhật khi xóa đối tượng

Trường hợp	**Đặc điểm**	**Kết quả**
Removal	Tập UpdSeedDel rỗng, không có đối tượng lõi còn lại nằm trong lân cận của những đối tượng vừa mất tính chất lõi.	Đối tượng bị xóa, một số đối tượng có thể trở thành nhiễu. Cụm có thể biến mất nếu không còn lõi.
Reduction	Các đối tượng thuộc UpdSeedDel đều khả đạt trực tiếp theo mật độ từ nhau sau phép xóa.	Cụm vẫn được duy trì nhưng có thể giảm số thành viên hoặc thay đổi tính chất của một số đối tượng.
Potential Split	Các đối tượng thuộc UpdSeedDel trước đó thuộc cùng một cụm, nhưng sau phép xóa không phải mọi cặp đối tượng đều kết nối trực tiếp theo mật độ.	Cần kiểm tra xem các đối tượng còn kết nối thông qua những lõi trung gian hay không, từ đó xác định Split hoặc Non-split.



Hình 2.2: Minh họa các trường hợp cập nhật khi xóa đối tượng: Removal, Reduction, Split và Non-split

Trong trường hợp Removal, tập UpdSeed_{Del} rỗng nên không cần thực hiện tìm kiếm kết nối giữa các đối tượng lõi khởi đầu. Đối với Reduction, các đối tượng thuộc tập này vẫn kết nối trực tiếp với nhau sau phép xóa, do đó không cần tìm đường nối thay thế.

Ngược lại, Potential Split xảy ra khi các đối tượng lõi khởi đầu không còn kết nối trực tiếp đầy đủ. Trong cài đặt của nhóm, thuật toán dùng tìm kiếm theo chiều rộng (BFS) để xác định xem các đối tượng này còn kết nối thông qua những lõi trung gian hay không. Nếu tồn tại đường nối thay thế, cụm được duy trì (Non-split), nếu không, cụm bị tách thành các thành phần riêng biệt (Split). Phần ví dụ chi tiết tập trung vào Potential Split để minh họa hai kết quả này (Ester et al., 1998; Massachusetts Institute of Technology, 2020).

Trường hợp 3 - Potential Split (Khả năng tách cụm)

Potential Split xảy ra khi phép xóa làm mất những kết nối trực tiếp cần thiết giữa các đối tượng lõi. Tuy nhiên, việc mất kết nối trực tiếp chưa đủ để kết luận cụm bị tách, bởi vẫn có thể tồn tại đường kết nối thông qua những lõi khác.

Để xác định kết quả, cần kiểm tra các chuỗi kết nối giữa những đối tượng lõi còn lại. Có hai khả năng:

- **Split:** Không còn đường kết nối lõi giữa một số phần của cụm, cụm ban đầu tách thành nhiều cụm.
- **Non-split:** Vẫn tồn tại đường kết nối lõi thay thế, cụm không bị tách.

Trường hợp A - Split (Tách cụm)

Xét mười đối tượng trong không gian hai chiều với $\varepsilon = 1{,}1$ và MinPts $= 4$:

$$\begin{aligned} A &= (1;0), & B &= (0{,}2;0), \\ C &= (0{,}6;0{,}6), & D &= (0{,}6;\text{-}0{,}6), \\ E &= (3;0), & F &= (3{,}8;0), \\ G &= (3{,}4;0{,}6), & H &= (3{,}4;\text{-}0{,}6), \\ I &= (2;1), & p &= (2;0). \end{aligned} \tag{2.25}$$

Trước khi xóa, A, B, E, F và p là các đối tượng lõi. Trong đó, p nằm giữa hai vùng đối tượng và có lân cận:

$$N_\varepsilon(p) = \{A, E, I, p\}. \tag{2.26}$$

Do p là lõi, nó tạo ra kết nối mật độ giữa hai phía của cụm thông qua các đối tượng A và E. Toàn bộ mười đối tượng ban đầu thuộc cùng một cụm.

Sau khi xóa p, các đối tượng A và E vẫn là lõi với số lượng lân cận giảm từ 5 xuống 4. Tuy nhiên, khoảng cách giữa chúng là:

$$d(A, E) = 2 > \varepsilon. \tag{2.27}$$

Vì vậy, A và E không còn kết nối trực tiếp. Đồng thời, không tồn tại chuỗi đối tượng lõi thay thế nối hai phía sau khi loại bỏ p.

Đối tượng I cũng không còn nằm trong lân cận của lõi nào và chuyển thành nhiễu.

Kết quả, cụm ban đầu tách thành hai cụm:

$$C_1 = \{A, B, C, D\}, \tag{2.28}$$

$$C_2 = \{E, F, G, H\}, \tag{2.29}$$

và tập nhiễu:

$$N = \{I\}. \tag{2.30}$$

Ví dụ cho thấy một đối tượng lõi có thể đóng vai trò nối hai phần của cùng một cụm. Khi đối tượng đó bị xóa và không còn chuỗi kết nối lõi thay thế, hai phần sẽ trở thành những cụm riêng biệt.

Trường hợp B - Non-split (Không tách cụm)

Sử dụng dữ liệu của trường hợp A và bổ sung ba đối tượng:

$$J = (1;1), \qquad K = (3;1), \qquad L = (2;2). \tag{2.31}$$

Các tham số $\varepsilon = 1{,}1$ và MinPts $= 4$ được giữ nguyên. Trước cập nhật, toàn bộ 13 đối tượng thuộc cùng một cụm.

Sau khi xóa p, đối tượng I vẫn là lõi vì lân cận còn bốn đối tượng là I, J, K và L. Đồng thời, J và K cũng tiếp tục thỏa mãn điều kiện lõi.

Mặc dù kết nối trực tiếp thông qua p không còn, thuật toán xác định được chuỗi lõi thay thế:

$$A \to J \to I \to K \to E. \tag{2.32}$$

Khoảng cách giữa từng cặp đối tượng liên tiếp trong chuỗi đều bằng 1, không vượt quá ε. Hơn nữa, tất cả đối tượng trong chuỗi đều là lõi sau cập nhật, nên các quan hệ khả đạt trực tiếp theo mật độ vẫn được duy trì dọc theo chuỗi.

Vì vậy, hai phía của cụm vẫn có quan hệ kết nối theo mật độ. Kết quả là 12 đối tượng còn lại tiếp tục thuộc cùng một cụm.

Hai ví dụ Split và Non-split cho thấy việc mất một đối tượng lõi không tự động dẫn đến tách cụm. Yếu tố quyết định là sau phép xóa có còn tồn tại đường kết nối thông qua những đối tượng lõi khác hay không (Ester et al., 1998).

Trong code, BFS bắt đầu từ các điểm trong UpdSeed$_{Del}$ và chỉ đi qua những điểm lõi còn kề nhau. Nếu đã tìm thấy đường nối giữa các seed thì chương trình dừng việc kiểm tra tách cụm. Nếu không tìm thấy đường nối, chương trình xác định các phần lõi bị tách và cập nhật nhãn của chúng. Như vậy, phần cài đặt có dùng UpdSeed$_{Del}$; BFS là bước kiểm tra kết nối sau khi tạo tập seed.

Quy trình DELETE trong chương trình của nhóm

Phương thức `delete()` cập nhật cụm quanh điểm bị xóa. Quy trình dưới đây theo

các bước mà nhóm đã cài đặt:

Bước 1. Tìm điểm p cần xóa và lưu thông tin lân cận, nhãn cụm, xác định các đối tượng lõi sẽ mất tính chất lõi sau khi xóa p.

Bước 2. Xóa p và bỏ ID của nó khỏi danh sách lân cận của các điểm còn lại.

Bước 3. Cập nhật trạng thái của các đối tượng vừa mất tính chất lõi, sau đó xác định tập UpdSeed_{Del} gồm những đối tượng vẫn là lõi và nằm trong lân cận của ít nhất một đối tượng vừa mất tính chất lõi.

Bước 4. Xét từng cụm liên quan. Nếu không có seed thì đó là Removal. Nếu các seed kề trực tiếp với nhau thì đó là Reduction và cụm không cần tách.

Bước 5. Nếu các seed không kề trực tiếp, dùng BFS để lần theo các điểm lõi. Nếu BFS nối được các seed thì cụm không tách (Non-split); nếu không, các phần lõi bị rời nhau được gán thành các cụm riêng (Split).

Bước 6. Cập nhật nhãn của các điểm lõi thuộc cụm bị tách, rồi phân loại lại các điểm không lõi liên quan thành biên hoặc nhiễu.

Các điểm biên không được dùng làm cầu nối giữa hai cụm. Vì vậy, chương trình xác định kết nối giữa các điểm lõi trước, rồi mới gán lại trạng thái và nhãn cho những điểm biên hoặc nhiễu bị ảnh hưởng.

Cách làm này cũng xử lý được trường hợp xóa một điểm biên hoặc xóa điểm mà không làm điểm lõi nào mất trạng thái lõi.

2.3.5 Khác biệt giữa Incremental DBSCAN và chạy lại DBSCAN

Incremental DBSCAN (cập nhật tăng dần) và chạy lại DBSCAN đều nhằm xác định cấu trúc phân cụm phù hợp với tập dữ liệu sau thay đổi. Điểm khác nhau nằm ở lượng thông tin được sử dụng lại và phạm vi cần xử lý.

Đối với cách chạy lại DBSCAN, mỗi lần cập nhật được xem như một bài toán phân cụm mới. Thuật toán thực hiện lại việc xác định lân cận, tính chất lõi và mở rộng cụm trên tập dữ liệu hiện tại.

Trong khi đó, Incremental DBSCAN giữ lại thông tin về đối tượng, lân cận, loại đối tượng và nhãn cụm sau mỗi lần xử lý. Khi phát sinh phép chèn hoặc xóa, thuật toán cập nhật những thông tin có thay đổi và điều chỉnh các cụm liên quan (Ester et al., 1998).

Bảng 2.5: So sánh cách cập nhật kết quả phân cụm

Tiêu chí	**Chạy lại DBSCAN**	**Incremental DBSCAN của nhóm**
Thông tin ban đầu	Phân cụm lại tập dữ liệu hiện tại.	Sử dụng kết quả phân cụm và lân cận đã lưu.
Phép chèn	Tính lại lân cận và mở rộng cụm trên toàn bộ dữ liệu.	Cập nhật lân cận của điểm mới và những đối tượng liên quan, sau đó xử lý các kết nối mới.
Phép xóa	Phân cụm lại toàn bộ dữ liệu còn lại.	Xác định các cụm liên quan, kiểm tra lại các thành phần lõi và gán lại nhãn.
Bộ nhớ	Phụ thuộc cách cài đặt và lưu trữ lân cận.	Cần duy trì tập lân cận và trạng thái của từng đối tượng.
Chi phí cập nhật	Phụ thuộc quy mô toàn bộ tập dữ liệu sau thay đổi.	Phụ thuộc phạm vi các cụm liên quan, chi phí duy trì trạng thái và những thao tác duyệt dữ liệu cần thiết để cập nhật nhãn cụm.

Nhờ lưu lại thông tin lân cận và trạng thái phân cụm, Incremental DBSCAN có thể tránh fit lại toàn bộ DBSCAN sau mỗi phép chèn hoặc xóa. Tuy nhiên, code hiện tại vẫn quét danh sách điểm khi tìm lân cận mới; khi hợp nhất hoặc tách cụm, chương trình cũng có thể phải duyệt nhiều điểm để cập nhật nhãn. Vì vậy, đây là cập nhật cục bộ về trạng thái, không bảo đảm mỗi thao tác chỉ cần xử lý một vùng nhỏ hoặc luôn chạy nhanh hơn.

Việc duy trì trạng thái cũng làm tăng nhu cầu lưu trữ so với những cách cài đặt không giữ lại toàn bộ thông tin lân cận. Vì vậy, Incremental DBSCAN không nhất thiết xử lý nhanh hơn DBSCAN chạy lại trong mọi tình huống. Lợi ích của phương pháp phụ thuộc vào phạm vi dữ liệu bị ảnh hưởng và lượng thông tin có thể tái sử dụng sau mỗi cập nhật.

2.4. Các chỉ số đánh giá

Nhóm xem kết quả theo ba khía cạnh: cập nhật có đúng không, các cụm có tách biệt không, và mất bao lâu để chạy. Silhouette là chỉ số chính để nhận xét chất lượng cụm; ARI dùng để đối chiếu mức đồng thuận giữa Incremental DBSCAN và các thuật toán chạy lại.

Nhóm cũng ghi nhận thời gian chạy, số cụm và tỷ lệ nhiễu để mô tả kết quả. Bộ nhớ được nhận xét qua lượng trạng thái thuật toán phải lưu; nhóm không đo bộ nhớ thành một thí nghiệm riêng.

2.4.1 Adjusted Rand Index (ARI)

Adjusted Rand Index (ARI) so sánh cách hai kết quả phân chia các cặp đối tượng vào cùng hoặc khác cụm, đồng thời hiệu chỉnh mức độ giống nhau có thể xuất hiện ngẫu nhiên (Hubert & Arabie, 1985).

Giá trị ARI được diễn giải như sau:

- ARI = 1: Hai kết quả có cùng cách phân nhóm, không phụ thuộc vào giá trị số của nhãn cụm.
- ARI ≈ 0: Mức độ tương đồng xấp xỉ giá trị kỳ vọng khi phân nhóm ngẫu nhiên.
- ARI < 0: Mức độ tương đồng thấp hơn giá trị kỳ vọng nói trên (Hubert & Arabie, 1985).

Trong phần kiểm tra tương đương, ARI đối chiếu Incremental DBSCAN với DBSCAN chạy lại trên cùng dữ liệu. Trong benchmark, chỉ số này còn ghi nhận mức đồng thuận giữa bản OOP với DBSCAN, HDBSCAN và OPTICS chạy lại. Nhãn nhiễu -1 được xem là một nhãn riêng. Vì ARI không thể hiện trạng thái lõi và biên, chương trình còn đối chiếu riêng các trạng thái này. ARI không được dùng để xếp hạng chất lượng cụm.

2.4.2 Silhouette

Silhouette cho biết một điểm gần với cụm của nó đến mức nào và cách các cụm khác bao xa. Chỉ số này không cần nhãn đáp án có sẵn (Rousseeuw, 1987).

Với mỗi điểm i, gọi $a(i)$ là khoảng cách trung bình đến các điểm khác trong cùng cụm, còn $b(i)$ là khoảng cách trung bình nhỏ nhất đến một cụm khác. Silhouette của điểm đó được tính như sau:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}. \tag{2.33}$$

Giá trị Silhouette nằm trong khoảng từ -1 đến 1, cụ thể như sau:

- Gần 1: Đối tượng nằm gần các thành viên cùng cụm và tách biệt tương đối rõ với những cụm khác.
- Gần 0: Đối tượng có mức độ gần tương tự với cụm hiện tại và một cụm khác, cho thấy sự phân biệt giữa các cụm chưa rõ ràng.
- Gần -1: Đối tượng có xu hướng nằm gần một cụm khác hơn so với cụm hiện tại, có thể cho thấy việc gán cụm chưa phù hợp (Rousseeuw, 1987).

Silhouette của toàn bộ kết quả phân cụm được xác định bằng cách lấy trung bình giá trị Silhouette của các đối tượng được đưa vào đánh giá. Giá trị trung bình càng cao thường cho thấy các cụm càng tập trung và tách biệt rõ theo khoảng cách sử dụng (Rousseeuw, 1987).

Cách nhóm tính Silhouette

Nhóm tự cài phép tính Silhouette theo công thức trên. Các điểm nhiễu có nhãn -1 được bỏ khỏi phép tính, còn các điểm thuộc cụm được giữ lại.

Chỉ tính điểm tổng hợp khi sau khi bỏ nhiễu còn ít nhất hai cụm. Nếu điều kiện này không đạt, kết quả được ghi là chưa xác định. Nếu một cụm chỉ có một điểm, Silhouette của điểm đó được quy ước bằng 0. Vì bỏ nhiễu có thể làm điểm trung bình tăng, nhóm đọc Silhouette cùng số cụm và tỷ lệ nhiễu.

Giới hạn khi đánh giá DBSCAN bằng Silhouette

Silhouette phản ánh sự gần nhau và tách biệt giữa các cụm dựa trên khoảng cách trung bình, nên thường thuận lợi cho những cụm có các đối tượng phân bố tập trung và cách xa nhau. Trong khi đó, DBSCAN có thể xác định những cụm có hình dạng không lồi. Với những dạng phân bố này, các đối tượng trong cùng cụm có thể cách nhau xa về mặt hình học nhưng vẫn được kết nối theo mật độ.

Vì vậy, Silhouette thấp không nhất thiết cho thấy DBSCAN phân cụm sai. Một kết quả phản ánh đúng các vùng kết nối theo mật độ vẫn có thể nhận Silhouette thấp hơn cách phân chia thành các cụm nhỏ, gọn hơn (scikit-learn developers, n.d.-a).

Ngoài ra, khi loại nhiều đối tượng nhiễu khỏi phép tính, Silhouette trung bình có thể tăng do chỉ còn các đối tượng được gom vào những cụm rõ ràng hơn. Do đó, nhóm cần báo cáo Silhouette cùng số lượng hoặc tỷ lệ nhiễu để tránh đánh giá kết quả chỉ dựa

trên những đối tượng được giữ lại.

2.4.3 Thời gian chạy và bộ nhớ

Thời gian thực hiện là một tiêu chí để đánh giá hiệu quả xử lý của Incremental DBSCAN. Khi so sánh với DBSCAN chạy lại, cần phân biệt thời gian phân cụm ban đầu và thời gian xử lý từng phép cập nhật. Chi phí cập nhật có thể thay đổi tùy theo loại thao tác và phạm vi cụm bị ảnh hưởng, do đó Incremental DBSCAN không nhất thiết nhanh hơn trong mọi trường hợp.

Bên cạnh thời gian xử lý, việc duy trì thông tin lân cận và trạng thái phân cụm cũng làm tăng nhu cầu bộ nhớ. Đây là sự đánh đổi giữa việc lưu trữ thông tin để tái sử dụng và việc phải tính toán lại từ đầu. Trong phạm vi đồ án, nhóm tập trung đo thời gian thực nghiệm, còn chi phí bộ nhớ được xem xét thông qua đặc điểm lưu trữ của thuật toán.

2.4.4 Số cụm và số điểm nhiễu

Số lượng cụm và số đối tượng nhiễu là những chỉ số mô tả trực tiếp kết quả phân cụm. Đối với Incremental DBSCAN, hai đại lượng này còn giúp theo dõi sự thay đổi cấu trúc dữ liệu sau mỗi phép chèn hoặc xóa.

Số cụm được xác định bằng cách đếm số nhãn cụm phân biệt trong kết quả, không tính nhãn -1 biểu thị nhiễu. Chẳng hạn, nếu tập dữ liệu có các nhãn 2, 2, 5, 5, -1, kết quả gồm hai cụm và một đối tượng nhiễu. Số cụm được xác định dựa trên số nhãn phân biệt, không phụ thuộc vào giá trị cụ thể của các nhãn.

Số đối tượng nhiễu là số đối tượng mang nhãn -1 trong tập dữ liệu hiện tại. Tỷ lệ nhiễu cho biết tỷ lệ đối tượng không được gán vào cụm nào.

Đối với các phép cập nhật, nhóm theo dõi số cụm và số đối tượng nhiễu trước và sau thao tác để xác định những thay đổi trong kết quả phân cụm. Tuy nhiên, sự tăng hoặc giảm số cụm chưa đủ để xác định cụ thể trường hợp cập nhật đã xảy ra. Chẳng hạn, số cụm giảm có thể do hợp nhất hai cụm sau phép chèn hoặc do một cụm biến mất hoàn toàn sau phép xóa.

Tương tự, số đối tượng nhiễu tăng hoặc giảm có thể phản ánh sự thay đổi giữa trạng thái nhiễu và biên khi mật độ lân cận thay đổi. Vì vậy, nhóm kết hợp các đại lượng này với thông tin về loại đối tượng và quan hệ thành viên cụm để phân tích từng kịch bản.

Đối với các bộ dữ liệu mô phỏng có đối tượng nhiễu được xác định trước, số lượng

và tỷ lệ nhiễu còn hỗ trợ đánh giá khả năng nhận diện các đối tượng nằm ngoài cấu trúc cụm.

2.5. Các thuật toán dùng để so sánh

Nhóm chọn DBSCAN, OPTICS và HDBSCAN để so sánh vì cả ba đều dựa trên mật độ nhưng tìm cụm theo những cách khác nhau. DBSCAN là đối chứng gần nhất với thuật toán nhóm tự cài. OPTICS và HDBSCAN giúp xem kết quả của hai cách phân cụm mật độ khác. Nhóm không đưa K-Means vào bảng so sánh chính vì K-Means thuộc cách phân cụm khác và không xử lý nhiễu như DBSCAN (Ankerst et al., 1999; Campello et al., 2013; Ester et al., 1996; scikit-learn developers, n.d.-a).

DBSCAN sử dụng hai tham số ε và MinPts để xác định các đối tượng lõi và mở rộng cụm. OPTICS xây dựng thứ tự xử lý cùng khoảng cách khả đạt (Reachability Distance), qua đó hỗ trợ xem xét cấu trúc cụm ở nhiều ngưỡng mật độ. Trong khi đó, HDBSCAN xây dựng cấu trúc phân cấp dựa trên mật độ và lựa chọn những cụm có mức ổn định phù hợp, giúp nhận diện các cụm có mật độ khác nhau trong nhiều trường hợp (Ankerst et al., 1999; Campello et al., 2013; scikit-learn developers, n.d.-a).

Trong benchmark, DBSCAN, HDBSCAN và OPTICS được lấy từ scikit-learn; sau mỗi cập nhật, các bản này được chạy lại trên toàn bộ dữ liệu hiện có. scikit-learn không cung cấp thao tác INSERT/DELETE tăng dần cho DBSCAN. Vì vậy, chỉ Incremental DBSCAN thực hiện cập nhật cục bộ, còn ba thuật toán đối chứng được chạy lại sau mỗi lần dữ liệu thay đổi, chứ không phải so sánh bốn thuật toán incremental (scikit-learn developers, n.d.-b, n.d.-c, n.d.-d).

Incremental DBSCAN vẫn dùng nguyên tắc mật độ của DBSCAN, nhưng tận dụng thông tin lân cận và cụm đã lưu để cập nhật sau INSERT hoặc DELETE.

Bảng 2.6: So sánh đặc điểm của các thuật toán phân cụm

Thuật toán	**Nguyên tắc phân cụm**	**Đặc điểm chính**
DBSCAN	Kết nối các đối tượng theo mật độ với ε và MinPts.	Nhận diện cụm không lồi và nhiễu; khó xử lý các cụm có mật độ chênh lệch lớn.
Incremental DBSCAN	Tận dụng kết quả DBSCAN đã có để cập nhật cấu trúc cụm sau INSERT hoặc DELETE.	Hỗ trợ chèn, xóa và sử dụng lại trạng thái, vẫn có thể cần duyệt nhiều đối tượng khi cập nhật nhãn cụm.
OPTICS	Sắp xếp các đối tượng theo cấu trúc mật độ và khoảng cách khả đạt.	Hỗ trợ phân tích cấu trúc cụm ở nhiều ngưỡng mật độ.
HDBSCAN	Xây dựng và lựa chọn cụm từ cấu trúc phân cấp theo mật độ.	Có thể nhận diện các cụm có mật độ khác nhau và xác định nhiễu.

CHƯƠNG 3: TỔNG QUAN BỘ DỮ LIỆU

3.1. Nguồn gốc và cách sử dụng dữ liệu

Các dữ liệu trong đề tài đều do nhóm mô phỏng bằng Python, không lấy từ khách hàng hay hệ thống ngân hàng thật. Nhóm dùng dữ liệu để minh họa DBSCAN trong không gian 2D, chạy các thao tác thêm/xóa và benchmark trên một luồng nhiều chiều.

Dữ liệu chính gồm tập tọa độ ATM 2D và luồng giao dịch thẻ 5D. Ngoài ra, nhóm chuẩn bị tám ví dụ nhỏ để minh họa các trường hợp cập nhật và một số ca nhỏ để kiểm thử tình huống biên.

3.2. Mô tả các bộ dữ liệu

Bảng 3.1: Các bộ dữ liệu được dùng trong đề tài

Bộ dữ liệu	**Quy mô**	**Số chiều**
ATM 2D	1.200 điểm	2
Luồng thẻ	500 điểm ban đầu, 300 cập nhật	5 thuộc tính gốc
Ví dụ cập nhật	8 kịch bản	2
Ca kiểm thử bổ sung	Một số ca nhỏ	1D/2D

ATM 2D. Tập này có 1.200 điểm, mỗi điểm gồm hai tọa độ `x_km` và `y_km`. Các tọa độ mô phỏng vị trí trong mặt phẳng, đơn vị kilômét. Cột `point_id` chỉ dùng để nhận diện điểm, không được đưa vào phép tính khoảng cách.

Luồng giao dịch thẻ 5D. Tập ban đầu D_0 có 500 giao dịch. Mỗi giao dịch có năm thuộc tính số:

Bảng 3.2: Các thuộc tính của luồng giao dịch thẻ

Thuộc tính	**Ý nghĩa mô phỏng**	**Đơn vị**
`amount_vnd`	Giá trị giao dịch	VNĐ
`hour_of_day`	Giờ thực hiện giao dịch trong ngày	Giờ
`distance_from_home_km`	Khoảng cách từ nơi giao dịch đến nhà	km
`txns_last_7d`	Số giao dịch trong bảy ngày trước đó	Lần
`merchant_risk_score`	Điểm rủi ro mô phỏng, từ 0 đến 1	Điểm

Cột `point_id` là mã định danh, không phải đặc trưng để phân cụm. Các giá trị được tạo để mô phỏng, nên không thể xem là mô tả hành vi của khách hàng ngoài đời.

Các ví dụ cập nhật và ca kiểm thử bổ sung

Tám ví dụ nhỏ minh họa Noise, Creation, Absorption, Merge, Removal, Reduction, Split và Non-split. Mỗi ví dụ có dữ liệu ban đầu, một thao tác cập nhật và tệp `expected.json` ghi kết quả mong đợi. Một số test dùng tọa độ ngắn hoặc fixture riêng để kiểm tra tình huống biên; đây là các ca chức năng nhỏ, không phải bộ dữ liệu dùng để báo cáo benchmark.

3.3. Tập ban đầu và các thao tác cập nhật

Tập D_0 được phân cụm trước bằng DBSCAN. Sau đó chương trình đọc lần lượt các dòng trong `updates.csv`: thao tác INSERT thêm một điểm, thao tác DELETE xóa một điểm đang có. File có tổng cộng 300 thao tác, gồm 150 INSERT và 150 DELETE xen kẽ; vì vậy số điểm trong luồng dao động từ 500 đến 501.

Thư mục `data/raw/card_stream/` chứa tập ban đầu, luồng cập nhật và file tham số. Trong `updates.csv`, cột `ambiguous_border_ids` là metadata tùy chọn cho bước đối chiếu các điểm biên; thuật toán không dùng cột này để quyết định cụm.

3.4. Các chiều dữ liệu trong benchmark

Benchmark 2D, 3D và 4D lấy các đặc trưng từ cùng luồng thẻ 5D, không phải ba bộ dữ liệu độc lập. Cấu hình 2D dùng số tiền đã lấy log và giờ giao dịch; 3D thêm khoảng cách từ nhà; 4D thêm số giao dịch trong bảy ngày. Điểm rủi ro của cửa hàng có trong dữ liệu gốc nhưng không dùng trong ba cấu hình này.

Các đặc trưng của thẻ được chuẩn hóa bằng những hằng số cố định lưu trong

`params.json`, rồi dùng cùng dữ liệu đã chuyển đổi cho các thuật toán được so sánh. Hằng số được tính từ toàn bộ luồng mô phỏng để giữ cách biến đổi nhất quán giữa các lượt; đây là giới hạn của thí nghiệm, vì khi chạy online thực tế ta thường chưa biết các giao dịch trong tương lai.

Luồng có 300 cập nhật, nhưng benchmark thời gian giữa bốn thuật toán chạy trên 20 cập nhật đầu để giữ thời gian thực nghiệm vừa phải. Vì 2D, 3D và 4D dùng cùng các điểm và cùng thứ tự cập nhật, kết quả giữa các chiều cho biết việc thêm đặc trưng ảnh hưởng thế nào trong thí nghiệm này; không nên xem chúng như ba lần thử độc lập.

3.5. Giới hạn của dữ liệu

Các bộ dữ liệu chỉ là mô phỏng và có quy mô nhỏ, nên kết quả dùng để minh họa và kiểm tra cách thuật toán hoạt động. Chúng chưa đủ để kết luận thuật toán sẽ có hiệu quả tương tự trên dữ liệu ngân hàng thật hoặc ở quy mô lớn.

CHƯƠNG 4: TIỀN XỬ LÝ VÀ CÀI ĐẶT

4.1. Môi trường và thư viện sử dụng

Chương trình được viết bằng Python và chia mã theo chức năng trong thư mục `src/`. Các phiên bản thư viện được khai báo trong `requirements.txt`. NumPy hỗ trợ thao tác với vector tọa độ, Pandas đọc và quản lý dữ liệu dạng bảng, còn Matplotlib được dùng để vẽ kết quả. Jupyter Notebook dùng để trình bày demo và benchmark.

DBSCAN và Incremental DBSCAN trong `src/clustering/` do nhóm tự cài đặt. Mã thuật toán không gọi hàm phân cụm của scikit-learn hay SciPy. scikit-learn chỉ được dùng trong phần đối chứng và đánh giá phụ; các hàm Silhouette chính của nhóm được cài riêng trong `src/evaluation/metrics.py`.

Bảng 4.1: Thư viện và mục đích sử dụng

Thành phần	**Mục đích trong đồ án**
Python	Cài đặt DBSCAN, Incremental DBSCAN, tiền xử lý và các bước thực nghiệm.
NumPy	Lưu và biến đổi vector số; hỗ trợ tính toán khoảng cách và trực quan hóa dữ liệu.
Pandas	Đọc CSV và quản lý dữ liệu dạng bảng.
Matplotlib	Trực quan hóa cụm, điểm lõi, điểm biên và điểm nhiễu.
scikit-learn	Chạy DBSCAN, HDBSCAN và OPTICS làm đối chứng; không được gọi trong mã thuật toán tự cài.
Jupyter	Trình bày demo từng bước và notebook so sánh thuật toán.
pytest	Chạy kiểm thử tự động cho các thao tác INSERT/DELETE, bao gồm các trường hợp cập nhật cụm, trạng thái lõi/biên/nhiễu và tính nhất quán của kết quả phân cụm.

4.2. Khảo sát dữ liệu

Trước khi chạy thuật toán, nhóm kiểm tra tên cột, số dòng, mã điểm và thứ tự các thao tác. Kết quả kiểm tra các file chính được tóm tắt ở Bảng 4.2. Chi tiết ý nghĩa thuộc tính đã trình bày trong Chương 3.

Bảng 4.2: Kiểm tra ban đầu các file dữ liệu được dùng

File	Số dòng	Kết quả kiểm tra
`atm_txn_2d.csv`	1.200	Có hai cột tọa độ dùng để phân cụm; không thiếu giá trị ở các cột sử dụng và không lặp mã điểm.
`card_stream/d0.csv`	500	Có năm thuộc tính số; không thiếu giá trị ở các cột sử dụng và không lặp mã điểm ban đầu.
`card_stream/updates.csv`	300	Có 150 thao tác INSERT và 150 thao tác DELETE; số bước chạy liên tục từ 1 đến 300.

Trong file cập nhật, các cột `step`, `op` và `point_id` dùng để điều khiển luồng; chúng không phải đặc trưng khoảng cách. Ô trống ở `ambiguous_border_ids` là thông tin phụ tùy chọn, không phải giá trị của một đặc trưng được đưa vào mô hình.

4.3. Làm sạch dữ liệu

Các file dùng trong demo là dữ liệu mô phỏng có cấu trúc xác định. Nhóm giữ nguyên các dòng dữ liệu, không tự điền giá trị thiếu và không loại điểm chỉ vì điểm đó nằm xa các điểm khác; những điểm này có thể chính là nhiễu mà DBSCAN cần nhận diện. Các mã định danh được giữ riêng, không đưa vào vector đặc trưng.

Khi đọc dữ liệu để chuẩn hóa, `ManualScaler` kiểm tra số chiều và từ chối giá trị không phải số, NaN hoặc vô cùng. Nếu một cột có độ lệch chuẩn hoặc khoảng biến thiên bằng 0, chương trình dùng hệ số chia bằng 1 để tránh phép chia cho 0. Việc kiểm tra này giúp phát hiện dữ liệu đầu vào không hợp lệ trước khi tính khoảng cách.

4.4. Chuẩn hóa dữ liệu

Tọa độ ATM có đơn vị kilômét, nên nhóm giữ nguyên hai tọa độ `x_km` và `y_km`; tham số Eps của demo ATM cũng được tính theo kilômét. Với luồng giao dịch thẻ, các thuộc tính có đơn vị và thang đo khác nhau. Nhóm lấy log cơ số 10 của số tiền giao dịch, sau đó chuẩn hóa Z-score theo công thức:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \tag{4.1}$$

trong đó x_j là giá trị của thuộc tính, μ_j là trung bình và σ_j là độ lệch chuẩn đã lưu trong `params.json`. Lớp `ManualScaler` tự tính và áp dụng phép biến đổi này, không dùng bộ chuẩn hóa của scikit-learn.

Các hằng số của luồng thẻ được giữ cố định khi xử lý D0 và các điểm cập nhật, để tọa độ của những điểm cũ không đổi sau mỗi thao tác. Chúng được tính từ toàn bộ luồng mô phỏng, kể cả các bước về sau; vì vậy cách làm phù hợp để so sánh có cùng đầu vào, nhưng chưa mô phỏng một hệ thống trực tuyến không biết dữ liệu tương lai.

4.5. Thiết lập tập ban đầu và luồng cập nhật

Với luồng thẻ, nhóm đọc 500 điểm trong D_0, chọn đúng các cột đặc trưng của cấu hình, áp dụng phép biến đổi đã lưu rồi dùng DBSCAN để tạo trạng thái ban đầu. Chương trình tiếp tục đọc `updates.csv` theo cột `step`. Mỗi dòng INSERT thêm một điểm vào mô hình; mỗi dòng DELETE xóa điểm có mã tương ứng.

Benchmark so sánh bốn phương pháp trên cùng D_0, tọa độ đã chuẩn hóa và thứ tự cập nhật. Nó chạy 20 thao tác đầu trong luồng 300 bước, gồm 10 INSERT và 10 DELETE. Incremental DBSCAN cập nhật trạng thái đã lưu; ba thuật toán scikit-learn được chạy lại trên toàn bộ tập hiện có sau mỗi thao tác. Cách tổ chức này giúp tách rõ khác biệt giữa cập nhật cục bộ và phân cụm lại.

4.6. Lựa chọn tham số eps và MinPts

Eps là bán kính xác định lân cận; MinPts là số điểm tối thiểu trong lân cận để một điểm trở thành điểm lõi. Lân cận có tính chính điểm đang xét. Các cấu hình trong đồ án khác nhau theo từng mục đích, như Bảng 4.3.

Bảng 4.3: Các cấu hình Eps và MinPts trong đồ án

Dữ liệu / mục đích	**Eps**	**MinPts**	**Lý do sử dụng**
Ví dụ cập nhật 2D	1,1	4	Được chọn để tạo rõ các trường hợp Noise, Creation, Absorption, Merge, Removal, Reduction, Split và Non-Split
ATM 2D	0,5 km	4	Dùng với tọa độ tính bằng kilômét để minh họa các cụm trên mặt phẳng.
Luồng thẻ 2D/3D/4D	0,4 sau chuẩn hóa	6	Lấy từ `params.json` và giữ nguyên giữa các thuật toán, số chiều và các lượt benchmark.

Trên tập D_0 gồm 500 điểm đã chuẩn hóa, nhóm thử DBSCAN tự cài đặt với Eps = 0,25; 0,30; 0,35; 0,40; 0,45; 0,50; 0,60 và MinPts = 4, 6, 8. Kết quả được đánh giá dựa trên Silhouette, số cụm và tỷ lệ điểm nhiễu.

Với MinPts = 6, Eps = 0,25 cho Silhouette cao hơn một chút so với Eps = 0,40, nhưng tạo nhiều nhiễu hơn. Ngược lại, khi tăng Eps lên 0,60, số cụm trong cấu hình 2D và 3D giảm từ 3 xuống 2 do các nhóm được hợp nhất. Trong khoảng Eps từ 0,35 đến 0,50, kết quả phân cụm tương đối ổn định về số cụm, tỷ lệ nhiễu và Silhouette.

Khi cố định Eps = 0,40, ba giá trị MinPts = 4, 6, 8 cho kết quả gần nhau. Vì vậy, nhóm sử dụng Eps = 0,40 và MinPts = 6 làm cấu hình tham chiếu thống nhất cho các thực nghiệm. Đây là lựa chọn phù hợp với dữ liệu mô phỏng đang xét, không đồng nghĩa với bộ tham số tối ưu cho mọi tập dữ liệu.

Khi so sánh, các thuật toán dùng chung dữ liệu, khoảng cách Euclidean và mốc MinPts = 6, nhưng tên và ý nghĩa tham số không hoàn toàn giống nhau. Bảng 4.4 ghi cấu hình thực tế trong mã benchmark. HDBSCAN không dùng Eps; OPTICS tạo thứ tự theo mật độ rồi trích cụm theo kiểu DBSCAN tại Eps = 0,40. Do đó, đây là cách đặt tham số tham chiếu để so sánh trên cùng đầu vào, không có nghĩa các thuật toán được

cấu hình tương đương hoàn toàn (scikit-learn developers, n.d.-c, n.d.-d).

Bảng 4.4: Cấu hình thuật toán trong benchmark

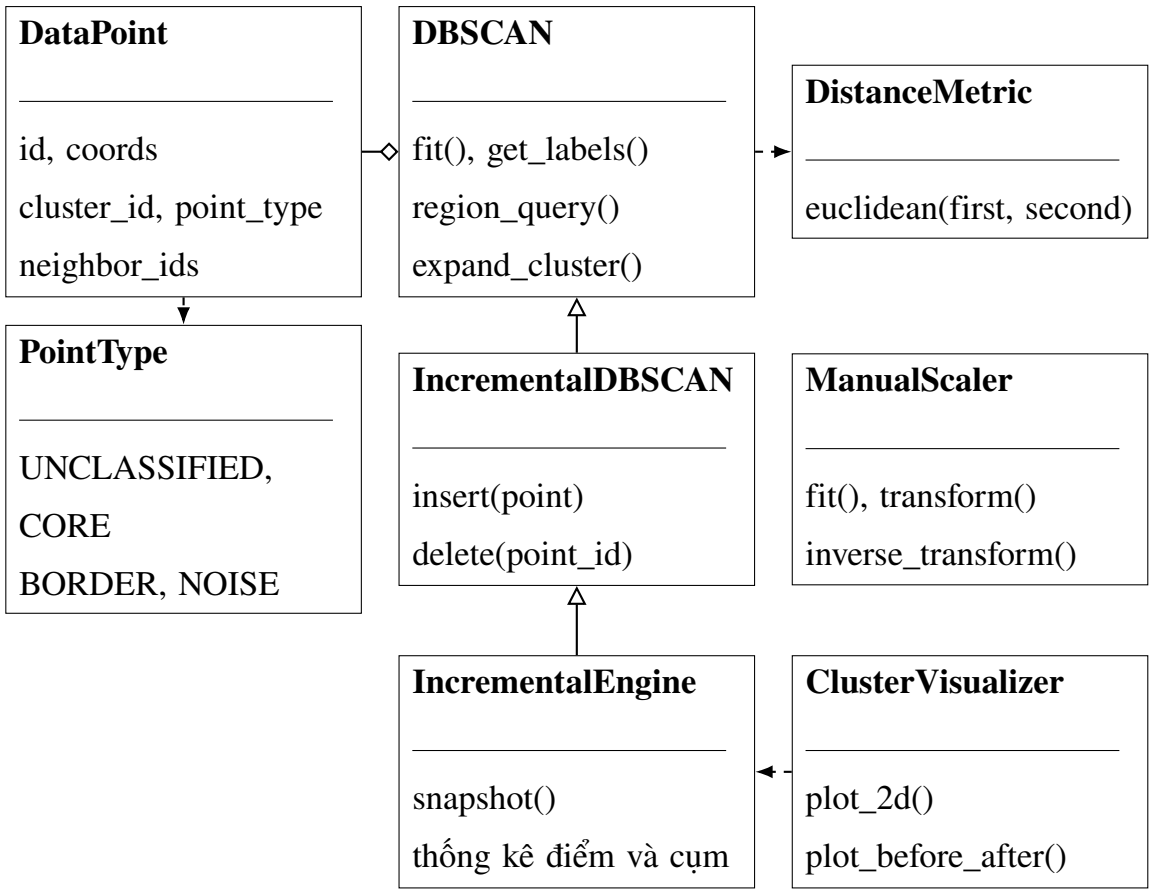
Thuật toán	**Tham số được dùng**	**Lý do cấu hình**
Incremental DBSCAN OOP	`eps=0.4`, `min_pts=6`, Euclidean	Tham số tham chiếu trong `params.json`; cập nhật trên trạng thái đã lưu.
scikit-learn DBSCAN	`eps=0.4`, `min_samples=6`, Euclidean	Dùng cùng ngưỡng lân cận và mốc số điểm với bản OOP; fit lại toàn bộ sau mỗi cập nhật.
scikit-learn HDBSCAN	`min_cluster_size=6`, `min_samples=6`, `cluster_selection_method=eom`	HDBSCAN không có Eps. Hai giá trị 6 làm mốc tham chiếu về kích thước cụm và mật độ; chúng không tương đương hoàn toàn với DBSCAN.
scikit-learn OPTICS	`min_samples=6`, `eps=0.4`, `max_eps=0.400001`, `cluster_method=dbscan`	Trích cụm theo kiểu DBSCAN tại cùng Eps; `max_eps` đặt nhỉnh hơn Eps để xây dựng thứ tự lân cận.

Các estimator scikit-learn trong benchmark được đặt `n_jobs=1` để chạy tuần tự khi đo thời gian. Các cấu hình minh họa 2D và ATM phục vụ mục đích khác nên không so trực tiếp giá trị Eps với benchmark luồng thẻ.

4.7. Thiết kế hướng đối tượng và sơ đồ lớp

Mỗi lớp giữ một phần trách nhiệm riêng. `DataPoint` lưu mã điểm, tọa độ, trạng thái và tập láng giềng. `DistanceMetric` tính khoảng cách Euclidean. `DBSCAN` tạo cụm ban đầu, còn `IncrementalDBSCAN` kế thừa lớp này và bổ sung thao tác thêm, xóa điểm. `IncrementalEngine` cung cấp ảnh chụp trạng thái để các notebook và bộ vẽ nhận cùng

một dạng dữ liệu. `ManualScaler` xử lý chuẩn hóa, còn `ClusterVisualizer` vẽ kết quả.



Hình 4.1: Sơ đồ các lớp chính trong chương trình

Trong hình, hình thoi rỗng biểu thị DBSCAN quản lý các đối tượng DataPoint; mũi tên tam giác rỗng biểu thị kế thừa; đường nét đứt biểu thị lớp dùng hoặc đọc thông tin của thành phần khác.

Bảng 4.5: Trách nhiệm của các lớp và phần hỗ trợ

Thành phần	**Trách nhiệm**	**Mã nguồn**
DataPoint, PointType	Lưu thông tin và trạng thái của mỗi điểm.	`src/core/data_point.py`
DistanceMetric	Tính khoảng cách Euclidean giữa hai vector cùng số chiều.	`src/core/distance.py`
ManualScaler	Fit, lưu và áp dụng tham số chuẩn hóa.	`src/preprocessing/scaler.py`
DBSCAN	Fit dữ liệu ban đầu, tìm lân cận và mở rộng cụm.	`src/clustering/dbscan.py`
Incremental DBSCAN	Cập nhật lân cận và nhãn cụm khi insert/delete.	`src/clustering/incremental_dbscan.py`
Incremental Engine	Trả snapshot và thống kê trạng thái để demo, đánh giá và vẽ.	`src/experiments/engine.py`
ClusterVisualizer	Vẽ cụm, loại điểm và trạng thái trước/sau cập nhật.	`src/visualization/plotter.py`
Hàm đánh giá	Tính Silhouette; được tổ chức thành module hàm, không phải một class riêng.	`src/evaluation/metrics.py`

Sơ đồ thể hiện các lớp hiện có trong mã. Nhóm không tạo class riêng cho bộ đánh giá hay bộ điều phối thí nghiệm vì các thao tác đó hiện được thực hiện bằng module hàm và notebook; cách tổ chức này giữ thiết kế ở mức cần thiết cho đồ án.

4.8. Cài đặt DBSCAN và Incremental DBSCAN

DBSCAN tính các điểm trong vùng Eps của mỗi điểm bằng khoảng cách Euclidean. Vùng lân cận có tính cả điểm trung tâm. Nếu số điểm trong vùng đạt MinPts, điểm trung tâm là điểm lõi; thuật toán dùng ngăn xếp LIFO để mở rộng cụm qua các điểm lõi lân cận. Điểm được nối vào cụm nhưng không đủ MinPts được ghi nhận là điểm biên; điểm không được nối vào cụm nào là nhiễu (Ester et al., 1996, 1998).

Incremental DBSCAN kế thừa trạng thái do DBSCAN tạo và giữ cache lân

cận theo mã điểm. Khi insert, chương trình cập nhật lân cận, tìm NewCore, rồi tạo UpdSeed_{Ins} từ các lõi trong lân cận của những điểm vừa thành lõi. Phần cập nhật được giới hạn trong các lõi mới: chương trình duyệt chúng bằng stack LIFO và dùng nhãn của các lõi cũ trong tập seed để xác định tạo cụm, hấp thụ hay hợp nhất cụm. Nhờ đó, chương trình không chạy lại DBSCAN trên toàn bộ dữ liệu; cách xác định vùng và seed dựa trên bài báo Incremental DBSCAN (Ester et al., 1998).

Khi delete, chương trình tìm LostCore, gồm điểm bị xóa nếu nó là lõi và các điểm kề vừa mất điều kiện lõi. Từ lân cận của các điểm này, chương trình lập UpdSeed_{Del}. Nếu tập seed rỗng thì xử lý Removal; nếu các seed trực tiếp nối nhau thì xử lý Reduction. Với Potential Split, BFS dùng hàng đợi FIFO khởi tạo từ các seed để tìm đường qua các lõi còn lại. Nếu các seed được nối với nhau thì cụm không tách; nếu không, các thành phần lõi được gán lại thành cụm riêng (Ester et al., 1998; Massachusetts Institute of Technology, 2020).

Thuật ngữ "cập nhật cục bộ" ở đây nghĩa là chương trình không fit lại toàn bộ DBSCAN. Tuy vậy, `_region_query()` hiện vẫn quét danh sách điểm để tìm lân cận; thao tác gộp cụm hoặc gán lại nhãn cũng có thể phải duyệt nhiều điểm. Vì thế, cách cài đặt này chưa bảo đảm mỗi cập nhật nhanh hơn fit lại, phù hợp với kết quả benchmark ở Chương 5.

Các vector đầu vào không bị giới hạn ở hai tọa độ. Khoảng cách và truy vấn lân cận dùng toàn bộ số chiều của vector; 2D được dùng cho ví dụ dễ nhìn, còn các lát 2D, 3D và 4D của luồng thẻ được phân cụm theo toàn bộ đặc trưng trong cấu hình tương ứng.

4.9. Cài đặt đánh giá và trực quan hóa

Nhóm tự cài Silhouette trong `src/evaluation/metrics.py`. Với mỗi điểm không phải nhiễu, chương trình tính khoảng cách trung bình đến điểm cùng cụm và khoảng cách trung bình nhỏ nhất đến một cụm khác, sau đó lấy trung bình điểm Silhouette. Điểm nhiễu được bỏ khỏi phép tính; điểm ở cụm đơn lẻ nhận giá trị 0. Khi dữ liệu sau khi bỏ nhiễu còn ít hơn hai cụm, hàm báo không thể tính Silhouette thay vì trả một kết quả gây hiểu nhầm. Công thức gốc của chỉ số được trình bày trong Chương 2 (Rousseeuw, 1987).

Phần đối chiếu gọi DBSCAN của scikit-learn để fit lại toàn bộ dữ liệu sau mỗi thao tác, rồi kiểm tra cách chia cụm và trạng thái điểm theo API của thư viện (scikit-

learn developers, n.d.-b). Đây là phần tham khảo độc lập, không tham gia vào mã thuật toán tự cài. Các hình được vẽ bằng Matplotlib trên dữ liệu hai chiều.

Hai notebook được sắp theo mục đích: `Incremental_DBSCAN_Demo.ipynb` đi qua ví dụ INSERT/DELETE và demo ATM 2D; `So_sanh_thuat_toan.ipynb` trình bày Silhouette và benchmark với DBSCAN, HDBSCAN, OPTICS. Cách chia này giúp người xem theo dõi thuật toán trước rồi mới xem phần so sánh.

4.10. Kiểm thử tự động

Nhóm dùng `pytest` để kiểm tra 10 tình huống chính của Incremental DBSCAN. Bảng 4.6 tóm tắt các tình huống INSERT và DELETE cần theo dõi khi trình bày thuật toán.

Bảng 4.6: Các tình huống kiểm thử chính của Incremental DBSCAN

Thao tác	**Số ca**	**Tình huống được kiểm tra**
INSERT	6	Noise, Creation, Absorption, Merge, điểm biên nằm gần hai cụm nhưng không làm chúng nhập lại, và một chuỗi chèn liên tiếp.
DELETE	4	Reduction, Removal, Split có cập nhật lại điểm biên, và Non-split khi vẫn còn đường nối thay thế.

Với INSERT, chương trình được kiểm tra qua loại điểm mới, số cụm, điểm lõi và tập seed cập nhật. Với DELETE, chương trình được kiểm tra qua số cụm còn lại, trạng thái điểm lõi/biên/nhiễu và phân hoạch sau khi xóa. Một test chèn liên tiếp nhiều điểm được giữ để xem các cập nhật có tiếp tục đúng trên trạng thái đã thay đổi hay không.

Bộ kiểm thử tập trung vào 10 tình huống INSERT/DELETE trong bảng trên, không mở rộng sang các phép kiểm tra phụ. Đây là các kiểm tra chức năng ở quy mô nhỏ; kết quả đối chiếu luồng dữ liệu và benchmark được trình bày ở Chương 5. Có thể chạy bằng lệnh `python -m pytest tests/`.

CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Thiết lập thực nghiệm

Nhóm thực hiện ba loại kiểm tra: chạy các ví dụ nhỏ để quan sát từng bước INSERT/DELETE, đối chiếu kết quả với DBSCAN chạy lại, và đo chất lượng cụm cùng thời gian cập nhật trên luồng thẻ mô phỏng.

Bảng 5.1: Các cấu hình chính trong thực nghiệm

Thí nghiệm	**Dữ liệu**	**Cấu hình**
Ví dụ IN-SERT/DELETE	Các kịch bản 2D trong Chương 2	Eps = 1,1; MinPts = 4
Minh họa ATM	1.200 tọa độ mô phỏng	Eps = 0,5 km; MinPts = 4
So sánh trên luồng thẻ	D0 có 500 điểm; benchmark phát lại 20 cập nhật đầu	Eps = 0,4; MinPts = 6; 2D, 3D và 4D

DBSCAN và Incremental DBSCAN OOP là phần nhóm tự cài. DBSCAN, HDB-SCAN và OPTICS của scikit-learn được dùng làm đối chứng (scikit-learn developers, n.d.-b, n.d.-c, n.d.-d). Trong benchmark luồng thẻ, bốn phương pháp nhận cùng dữ liệu đã chuẩn hóa, khoảng cách Euclidean và thứ tự cập nhật. Ba thuật toán thư viện được chạy lại trên toàn bộ dữ liệu hiện có sau mỗi cập nhật; bản Incremental DBSCAN xử lý INSERT/DELETE trên trạng thái đã lưu. Các tham số riêng của từng phương pháp được trình bày ở Chương 4, Bảng 4.4. Demo ATM 2D dùng tọa độ kilômét chưa chuẩn hóa và cấu hình riêng.

Benchmark thời gian chạy 5 lượt, có một lượt khởi động không tính vào kết quả. Mỗi lượt dùng cùng dữ liệu và cùng thứ tự 20 cập nhật, gồm 10 INSERT và 10 DELETE. Bảng thời gian báo trung vị và độ lệch chuẩn. Silhouette được tính trên trạng thái cuối của 20 cập nhật trong một lần chạy xác định, không phải trung bình của 5 lượt thời gian.

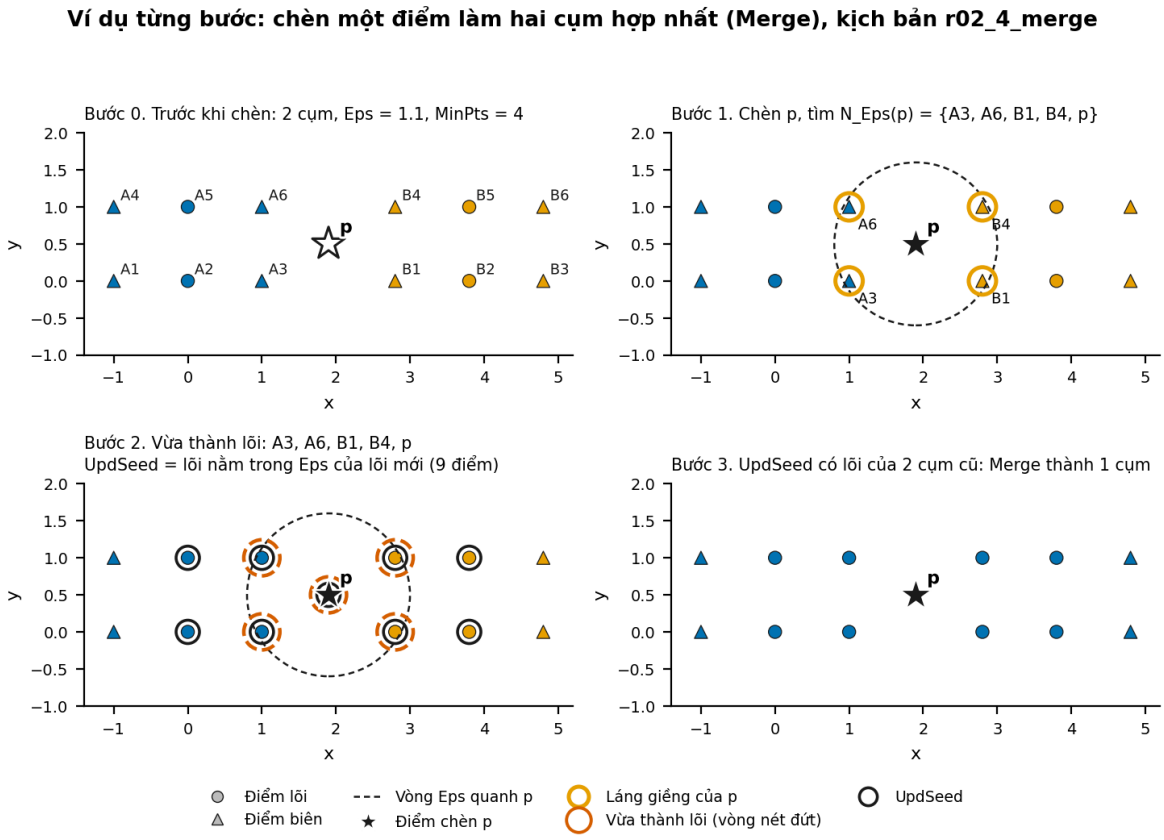
5.2. Kết quả khi thêm điểm

Các ví dụ INSERT dùng Eps $=$ 1,1, MinPts $=$ 4. Bốn trường hợp cho thấy một điểm mới có thể là nhiễu, tạo cụm mới, gia nhập cụm đang có hoặc làm các cụm nhập lại với nhau (Ester et al., 1998).

Bảng 5.2: Kết quả các trường hợp INSERT

Trường hợp	**Điều xảy ra**	**Kết quả sau cập nhật**
Noise	Điểm mới không đủ lân cận và không nằm cạnh điểm lõi.	Điểm mới được đánh dấu nhiễu; cụm cũ giữ nguyên.
Creation	Điểm mới làm một số điểm đạt điều kiện lõi, nhưng chưa nối tới cụm cũ.	Các điểm liên quan tạo thành cụm mới.
Absorption	Điểm mới hoặc các điểm lân cận mới đạt điều kiện lõi trong vùng của một cụm cũ.	Cụm cũ được mở rộng; một số điểm nhiễu có thể thành điểm biên.
Merge	Các điểm lõi mới tạo đường nối giữa hai cụm cũ.	Các cụm có liên quan được gộp thành một cụm.

Trong ví dụ Merge, dữ liệu ban đầu có hai cụm. Điểm p được chèn vào giữa và làm các điểm ở hai phía trở thành lõi. Tập UpdSeed_{Ins} chạm tới cả hai cụm, nên chương trình gộp chúng. Hình 5.1 thể hiện các bước từ trước khi chèn đến sau khi cập nhật.



Hình 5.1: Các bước chèn điểm làm hai cụm nhập lại thành một cụm

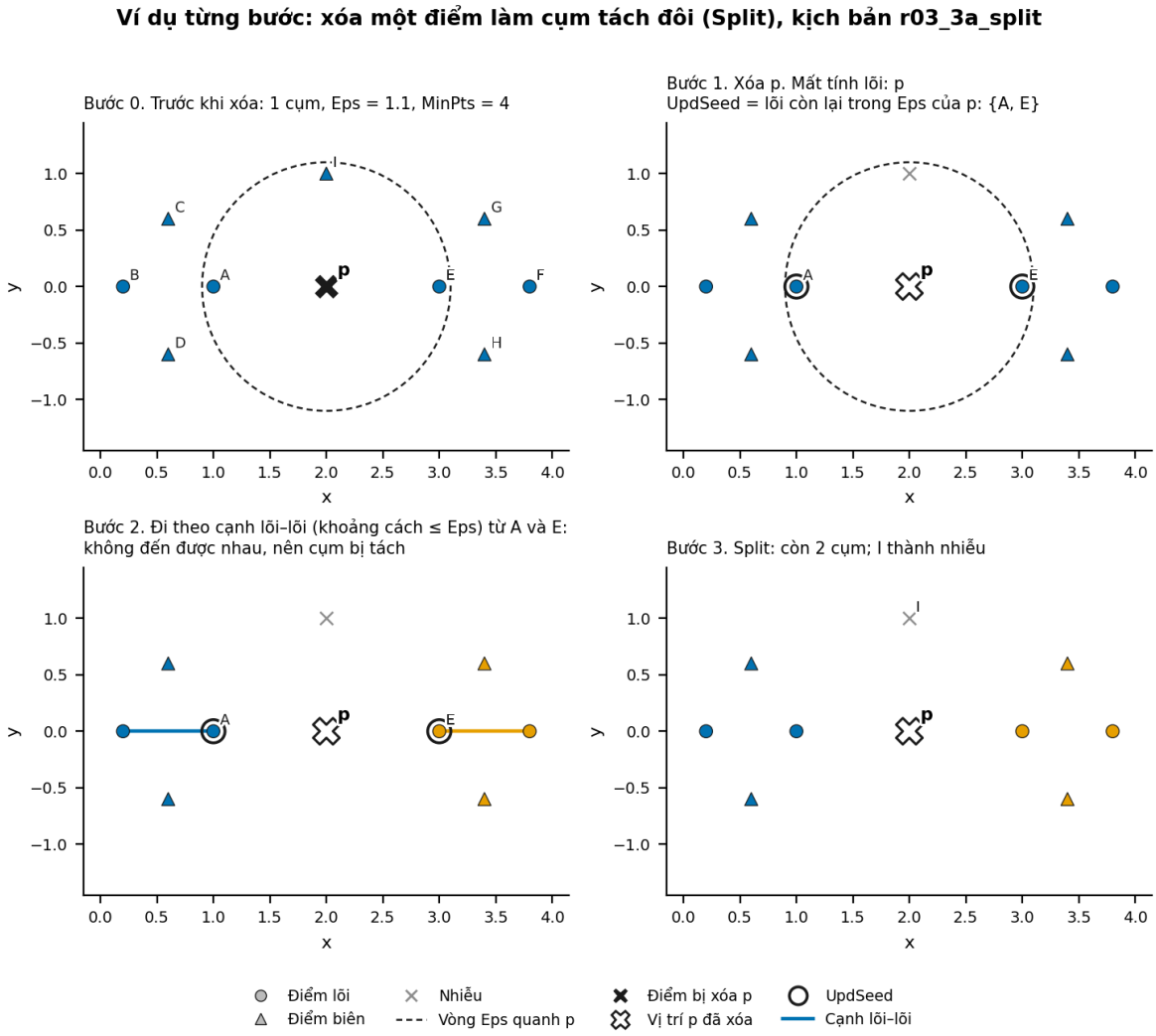
5.3. Kết quả khi xóa điểm

Các ví dụ DELETE cũng dùng Eps = 1,1, MinPts = 4. Xóa một điểm không phải lúc nào cũng làm cụm bị tách. Kết quả phụ thuộc vào việc các điểm lõi còn giữ được đường nối hay không (Ester et al., 1998).

Bảng 5.3: Kết quả các trường hợp DELETE

Trường hợp	**Điều xảy ra**	**Kết quả sau cập nhật**
Removal	Không còn seed lõi cần kiểm tra; có thể điểm bị xóa là lõi duy nhất của cụm.	Cụm có thể biến mất; các điểm còn lại thành nhiễu nếu không còn lõi nào.
Reduction	Các seed của cụm vẫn kề trực tiếp nhau.	Cụm không bị tách, dù một số điểm có thể mất trạng thái lõi và trở thành biên.
Potential Split - Split	BFS từ các seed không tìm được đường qua điểm lõi để nối các phía.	Cụm cũ tách thành nhiều cụm; điểm không còn cạnh lõi có thể thành nhiễu.
Potential Split - Non-split	Có đường nối thay thế qua các điểm lõi khác.	Cụm vẫn giữ nguyên, dù điểm cầu nối ban đầu đã bị xóa.

Ví dụ Split bắt đầu với một cụm có điểm p nối hai phía. Sau khi xóa p, BFS không tìm được đường qua các điểm lõi giữa hai phía, nên cụm tách làm hai và điểm I trở thành nhiễu. Hình 5.2 minh họa kết quả này.



Hình 5.2: Các bước xóa điểm cầu nối làm một cụm tách thành hai cụm

Ở trường hợp Non-split, sau khi xóa điểm cầu nối ban đầu vẫn còn một đường đi qua các điểm lõi. Vì vậy BFS xác nhận các seed vẫn thuộc cùng một thành phần và không tách cụm. Hai trường hợp này cho thấy chỉ biết một điểm lõi bị xóa là chưa đủ; cần kiểm tra các đường nối còn lại.

5.4. Minh họa trên không gian 2D

Các ví dụ Merge và Split ở trên dùng tọa độ 2D để có thể theo dõi trực tiếp điểm mới, điểm bị xóa, vòng Eps và các điểm trong UpdSeed. Notebook minh họa trình bày từng bước của hai ví dụ này, sau đó chạy DBSCAN trên 1.200 tọa độ ATM.

Với ATM 2D, cấu hình Eps = 0,5 km và MinPts = 4 tạo ra 6 cụm, 142 điểm nhiễu; Silhouette tự cài trên các điểm không nhiễu là 0,3362. Các phương pháp nhận cùng hai tọa độ và khoảng cách Euclidean. DBSCAN dùng `eps=0.5, min_samples=4`; HDBSCAN dùng `min_cluster_size=4, min_samples=4` vì thuật toán này không

có Eps; OPTICS dùng `min_samples=4, eps=0.5, max_eps=0.500001` và trích cụm theo kiểu DBSCAN. Vì Silhouette bỏ điểm nhiễu, nhóm đọc chỉ số này cùng số cụm và tỷ lệ nhiễu, không dùng riêng nó để kết luận mô hình nào tốt hơn.

5.5. Đối chiếu với DBSCAN chạy lại

Để kiểm tra phép cập nhật, nhóm so sánh trạng thái của Incremental DBSCAN với DBSCAN của scikit-learn được fit lại trên cùng tập điểm sau mỗi thao tác. Phép đối chiếu xét phân hoạch cụm, trạng thái lõi và điểm nhiễu; mã nhãn cụm có thể khác nhau nhưng cách chia điểm phải tương ứng.

Trong kết quả lưu tại `results/equivalence_results.csv`, có 317 lần đối chiếu: 9 lần fit ban đầu, 154 INSERT và 154 DELETE. Cả 317 lần đều đạt điều kiện tương đương; không có trường hợp sai khác được ghi nhận. Kết quả này gồm các ví dụ cập nhật và luồng thẻ, cho thấy các trạng thái đã kiểm tra của bản OOP khớp với DBSCAN chạy lại.

Đây là kiểm tra trên các bộ dữ liệu và thao tác hiện có, không chứng minh thuật toán đúng với mọi dữ liệu có thể gặp.

5.6. So sánh chất lượng cụm bằng Silhouette

Nhóm tự cài Silhouette và dùng chỉ số này làm cách đánh giá chất lượng cụm chính. Điểm nhiễu bị bỏ khỏi phép tính, nên bảng luôn kèm số cụm và tỷ lệ nhiễu. ARI dùng trong kiểm tra tương đương giữa hai cách cài DBSCAN; ở benchmark, ARI ghi nhận mức đồng thuận giữa bản Incremental DBSCAN OOP với DBSCAN, HDBSCAN và OPTICS chạy lại. ARI chỉ dùng để đối chiếu phân hoạch, không dùng để xếp hạng chất lượng cụm.

Trên ATM 2D, các phương pháp cho kết quả sau:

Bảng 5.4: So sánh Silhouette trên dữ liệu ATM 2D

Phương pháp	**Silhouette**	**Số cụm**	**Tỷ lệ nhiễu**
Incremental DBSCAN OOP	0,3362	6	11,83%
scikit-learn DBSCAN	0,3362	6	11,83%
scikit-learn HDBSCAN	0,3978	23	9,92%
scikit-learn OPTICS	0,3362	6	11,83%

HDBSCAN có Silhouette cao hơn trong cấu hình này nhưng đồng thời tạo ra 23 cụm, nhiều hơn đáng kể so với sáu cụm của các phương pháp còn lại. Vì vậy, không thể chỉ nhìn điểm Silhouette rồi kết luận HDBSCAN tốt hơn; cần xem số cụm, tỷ lệ nhiễu và mục tiêu của bài toán.

Trên luồng thẻ, bảng dưới tính tại trạng thái cuối sau 20 cập nhật. Cả bốn phương pháp cho cùng 3 cụm và 1% điểm nhiễu trong ba lát đặc trưng đang thử.

Bảng 5.5: Silhouette trên các lát 2D, 3D và 4D của luồng thẻ

Số chiều	Silhouette	Số cụm	Nhiễu	Ghi chú
2D	0,7785	3	1%	Bốn phương pháp cho cùng phân hoạch trong cấu hình này.
3D	0,7839	3	1%	Bốn phương pháp cho cùng phân hoạch trong cấu hình này.
4D	0,7463	3	1%	Bốn phương pháp cho cùng phân hoạch trong cấu hình này.

Các lát 2D, 3D và 4D được lấy từ cùng một luồng dữ liệu, nên không phải ba bộ dữ liệu độc lập. Kết quả giống nhau giữa bốn phương pháp chỉ áp dụng cho dữ liệu, tham số và 20 cập nhật này; không có nghĩa các thuật toán luôn cho cùng kết quả.

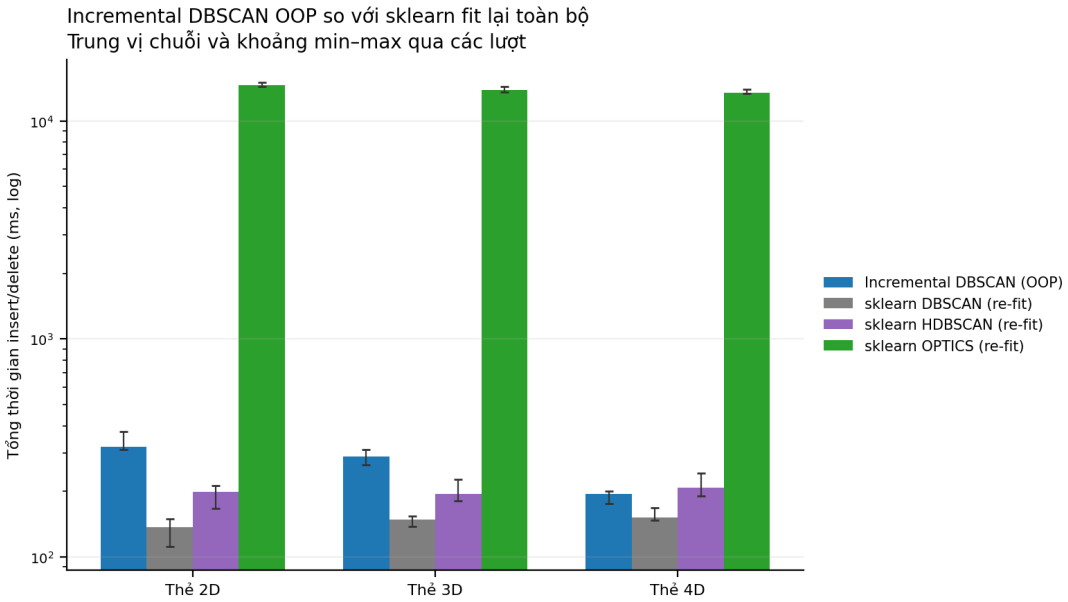
5.7. Thời gian chạy và giới hạn đo đạc

Bảng 5.6 báo tổng thời gian của 20 cập nhật, lấy trung vị ± độ lệch chuẩn qua 5 lượt. Đơn vị là mili giây.

Bảng 5.6: Thời gian cập nhật trên 20 thao tác

Dữ liệu	Incremental OOP	sklearn DBSCAN	sklearn HDBSCAN	sklearn OPTICS
Thẻ 2D	321,08 ± 25,53	136,72 ± 14,81	198,41 ± 18,84	14.571,56 ± 238,01
Thẻ 3D	288,78 ± 20,95	148,09 ± 6,57	194,18 ± 19,08	13.824,08 ± 373,53
Thẻ 4D	194,56 ± 9,98	152,03 ± 8,23	208,37 ± 22,12	13.485,85 ± 232,03

Trong phép đo này, DBSCAN scikit-learn fit lại có tổng thời gian thấp hơn bản OOP ở cả ba số chiều. HDBSCAN cũng nhanh hơn ở 2D và 3D, còn bản OOP thấp hơn một chút ở 4D. OPTICS có thời gian cao nhất trong cả ba cấu hình. Kết quả này không cho thấy Incremental DBSCAN luôn nhanh hơn cách fit lại; hình 5.3 thể hiện chênh lệch trên thang log.



Hình 5.3: Tổng thời gian cập nhật qua 5 lượt benchmark

Tuy vậy, thời gian fit ban đầu D0 của bản OOP cao hơn DBSCAN scikit-learn. Bảng 5.7 tách riêng bước này để tránh chỉ nhìn thời gian cập nhật rồi kết luận bản OOP nhanh hơn trong toàn bộ quá trình.

Bảng 5.7: Thời gian fit ban đầu D0, trung vị qua 5 lượt (ms)

Dữ liệu	**Incremental OOP**	**sklearn DBSCAN**	**sklearn HDBSCAN**	**sklearn OPTICS**
Thẻ 2D	1.315,57	8,22	10,53	697,82
Thẻ 3D	1.288,58	7,75	10,46	701,88
Thẻ 4D	1.434,08	9,42	13,32	693,52

Nhóm không đo bộ nhớ thành một thí nghiệm riêng. Phần này chỉ nhận xét định tính rằng Incremental DBSCAN phải lưu thêm lân cận và trạng thái điểm để dùng lại khi cập nhật. Thời gian nêu trên cũng chỉ phản ánh môi trường chạy, dữ liệu mô phỏng và 20 cập nhật đầu; không đại diện cho toàn bộ 300 cập nhật hay dữ liệu giao dịch thật.

5.8. Nhận xét chung

Các ví dụ nhỏ giúp quan sát rõ việc thêm điểm có thể tạo hoặc gộp cụm, còn xóa điểm có thể làm cụm giảm, biến mất hoặc tách ra. Kết quả đối chiếu hiện có không ghi nhận sai khác giữa Incremental DBSCAN và DBSCAN chạy lại. Trong benchmark thẻ, DBSCAN scikit-learn fit lại nhanh hơn bản OOP ở cả ba số chiều; HDBSCAN nhanh hơn ở 2D/3D và chậm hơn một chút ở 4D. Bước fit ban đầu của bản OOP cũng chậm hơn các phương pháp thư viện. Vì vậy, thí nghiệm hiện tại xác nhận tính đúng của cập nhật, nhưng chưa cho thấy lợi thế thời gian của bản OOP trên prefix 20 thao tác này.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Qua đề tài, nhóm đã tự cài DBSCAN và Incremental DBSCAN bằng Python theo hướng đối tượng. DBSCAN dùng để phân cụm ban đầu; phần Incremental DBSCAN hỗ trợ thêm và xóa điểm, sử dụng lại thông tin lân cận và trạng thái cụm. Các bước cập nhật được xây dựng theo những ý chính trong tài liệu Incremental DBSCAN, gồm UpdSeed_{Ins}, UpdSeed_{Del}, kiểm tra kết nối và xử lý trường hợp cụm bị tách (Ester et al., 1998).

Nhóm đã minh họa các trường hợp INSERT và DELETE bằng ví dụ 2D, đồng thời chạy DBSCAN trên tọa độ ATM và khảo sát luồng thẻ theo các lát 2D, 3D, 4D. Silhouette do nhóm tự cài được dùng làm chỉ số chính để nhận xét chất lượng cụm; số cụm và tỷ lệ nhiễu được xem kèm để tránh kết luận chỉ từ một con số.

Trong file đối chiếu hiện có, 317 trạng thái sau khi fit hoặc cập nhật đều tương đương với DBSCAN chạy lại. Trên benchmark 20 cập nhật, DBSCAN scikit-learn fit lại nhanh hơn bản OOP ở cả 2D, 3D và 4D; HDBSCAN nhanh hơn ở 2D/3D và chậm hơn một chút ở 4D. OPTICS chậm nhất trong cả ba cấu hình. Bước fit D0 ban đầu của bản OOP cũng lâu hơn các bản thư viện. Vì vậy, thí nghiệm này kiểm tra được tính đúng và cho thấy thời gian phụ thuộc vào cách cài đặt, nhưng chưa chứng minh Incremental DBSCAN OOP có lợi thế tốc độ trên luồng đang đo.

6.2. Hạn chế của đề tài

Các dữ liệu đều là mô phỏng, nên chưa cho biết thuật toán hoạt động thế nào trên giao dịch thật. Benchmark thời gian chỉ chạy 20 cập nhật đầu trong luồng 300 cập nhật; các chiều 2D, 3D và 4D lấy từ cùng dữ liệu nên không phải các thử nghiệm độc lập.

Các hằng số chuẩn hóa của luồng thẻ được tính từ toàn bộ dữ liệu mô phỏng, bao gồm cả các cập nhật ở phía sau. Cách này giúp giữ tọa độ nhất quán giữa các thuật toán để so sánh, nhưng chưa mô phỏng một hệ thống online chỉ biết dữ liệu đến hiện tại.

Ba phương pháp scikit-learn được fit lại toàn bộ sau mỗi cập nhật, vì vậy đây là đối chiếu với cách phân cụm lại chứ không phải với một thư viện Incremental DBSCAN. Thời gian phụ thuộc vào môi trường chạy và cách cài đặt. Nhóm chưa đo bộ nhớ, và bộ tham số cố định trong benchmark cũng không được xem là tối ưu cho mọi dữ liệu.

6.3. Hướng phát triển

Nếu có thêm thời gian và dữ liệu phù hợp, nhóm có thể tiếp tục theo các hướng sau:

- Thử trên dữ liệu đã ẩn thông tin cá nhân hoặc dữ liệu thực được phép sử dụng, rồi so sánh với kết quả trên dữ liệu mô phỏng.
- Phát lại nhiều bước cập nhật hơn và kiểm tra thêm các kích thước, số chiều để xem chi phí thay đổi ra sao.
- Tính tham số chuẩn hóa theo dữ liệu đã quan sát tại từng thời điểm để mô phỏng luồng online sát hơn.
- Đo thêm bộ nhớ và tìm cách giảm thời gian fit ban đầu, chẳng hạn bằng cấu trúc tra cứu lân cận phù hợp.
- Nếu tìm được một cài đặt Incremental DBSCAN có thể đối chiếu, so sánh thêm với cách cập nhật tăng dần đó thay vì chỉ so với các thuật toán fit lại toàn bộ.

Các hướng này là phần có thể làm tiếp, không nằm trong kết quả đã báo cáo ở đề tài hiện tại.

TÀI LIỆU THAM KHẢO

Ankerst, M., Breunig, M. M., Kriegel, H.-P., & Sander, J. (1999). Optics: Ordering points to identify the clustering structure. *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, 49–60. https://doi.org/10.1145/304182.304187

Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *Advances in Knowledge Discovery and Data Mining*, *7819*, 160–172. https://doi.org/10.1007/978-3-642-37456-2_14

Ester, M., Kriegel, H.-P., Sander, J., Wimmer, M., & Xu, X. (1998). Incremental clustering for mining in a data warehousing environment. *Proceedings of the 24th International Conference on Very Large Data Bases*, 323–333. https://www.vldb.org/conf/1998/p323.pdf

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226–231. https://cdn.aaai.org/KDD/1996/KDD96-037.pdf

Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, *2*(1), 193–218. https://doi.org/10.1007/BF01908075

Machine Learning Cơ Bản. (2017, January 1). *Bài 4: K-means clustering*. Retrieved November 4, 2026, from https://machinelearningcoban.com/2017/01/01/kmeans/

Massachusetts Institute of Technology. (2020). *6.006 introduction to algorithms: Lecture 9 - breadth-first search*. MIT OpenCourseWare. Retrieved November 4, 2026, from https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/lecture-9-breadth-first-search/

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

scikit-learn developers. (n.d.-a). *Clustering*. scikit-learn. Retrieved November 4, 2026, from https://scikit-learn.org/stable/modules/clustering.html

scikit-learn developers. (n.d.-b). *Dbscan*. scikit-learn. Retrieved November 4, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html

scikit-learn developers. (n.d.-c). *Hdbscan*. scikit-learn. Retrieved November 4, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.cluster.HDBSCAN.html

scikit-learn developers. (n.d.-d). *Optics*. scikit-learn. Retrieved November 4, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.cluster.OPTICS.html

PHỤ LỤC

A - BẢNG PHÂN CÔNG CÔNG VIỆC

Bảng A.1: Phân công công việc và tỷ lệ đóng góp của thành viên

Mã	Thành viên	Vai trò	% đóng góp
TV1	Nguyễn Hoàng Anh	Leader, DBSCAN nền tảng và tích hợp	100%
TV2	Lý Huỳnh Phương Anh	Lý thuyết Incremental DBSCAN và INSERT	100%
TV3	Từ Thiên Gia Bảo	Thiết kế OOP, DBSCAN và DELETE	100%
TV4	Nguyễn Phạm Gia Bảo	Dữ liệu, thực nghiệm và trực quan hóa	100%
TV5	Hồ Ngọc Bảo	Không tham gia đồ án	0%

B - CÔNG BỐ VIỆC SỬ DỤNG AI

Trong quá trình thực hiện đồ án, nhóm có sử dụng ChatGPT/Codex như công cụ hỗ trợ. Công cụ được dùng để đề xuất và chỉnh sửa một số đoạn mã, giải thích tài liệu thuật toán, biên tập nội dung báo cáo và hỗ trợ định dạng LaTeX.

AI không được dùng để thay thế các phép chạy thực nghiệm hoặc tạo ra các con số kết quả trình bày trong báo cáo. Các số liệu được lấy từ chương trình, notebook và các file kết quả trong repository. Nhóm chịu trách nhiệm kiểm tra mã nguồn, đối chiếu cách cài đặt với tài liệu tham khảo, xem lại kết quả chạy và chỉnh sửa nội dung trước khi nộp.

C - MÃ NGUỒN VÀ HƯỚNG DẪN CHẠY

Mã nguồn và dữ liệu của đề tài được lưu trong repository của nhóm: Incremental DBSCAN Nhóm 1. Các lệnh dưới đây được chạy từ thư mục gốc repository.

Trang thống kê người đóng góp của repository được xem tại GitHub Contributors.

1. Tạo môi trường Python và cài thư viện:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

2. Chạy demo ngắn INSERT/DELETE:

```
python run_demo.py
```

3. Mở notebook minh họa thuật toán và notebook so sánh:

```
jupyter notebook notebooks/Incremental_DBSCAN_Demo.ipynb
jupyter notebook notebooks/So_sanh_thuat_toan.ipynb
```

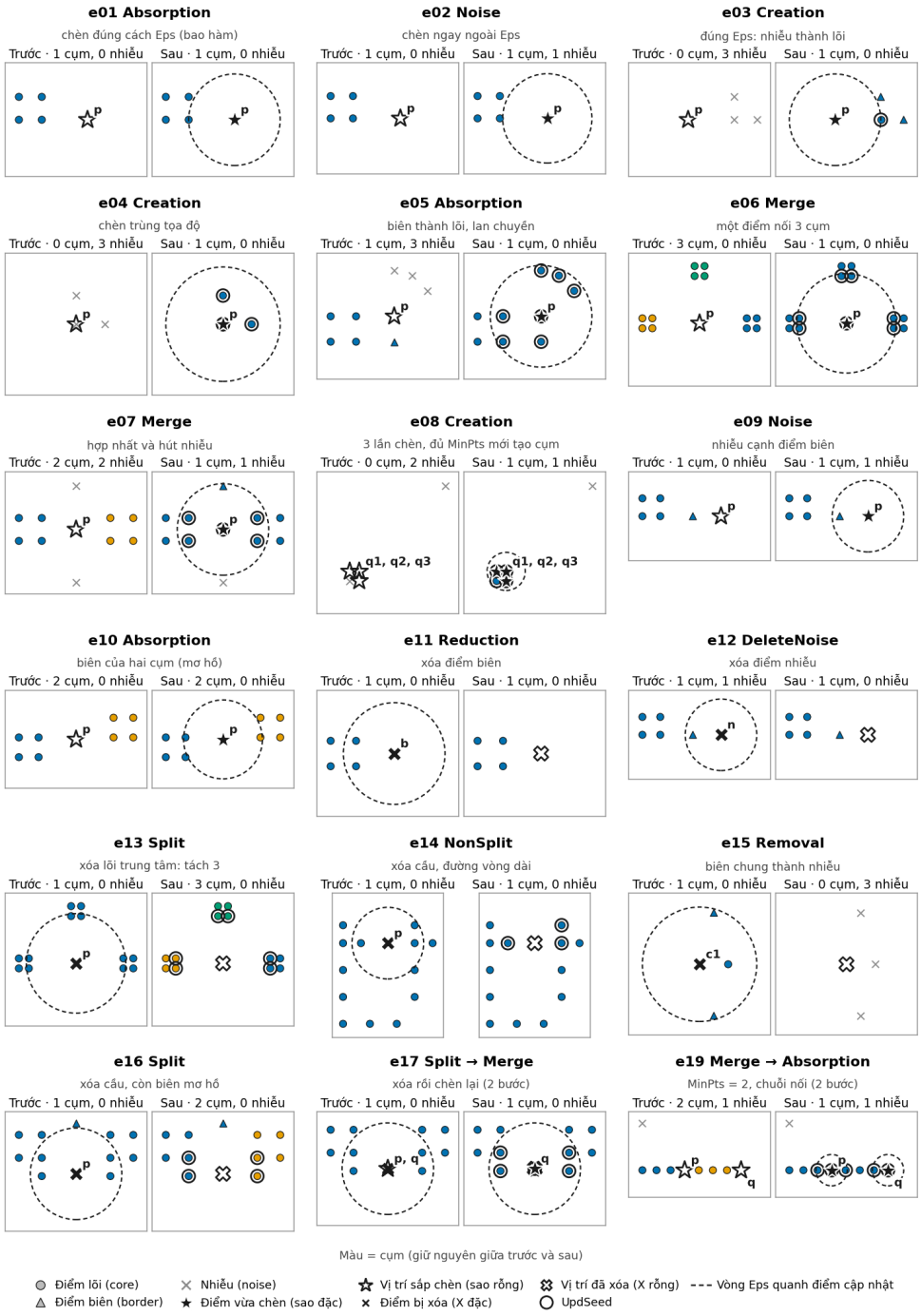
4. Chạy đối chiếu và kiểm tra tự động khi cần:

```
python -m src.evaluation.equivalence
python -m pytest tests/
```

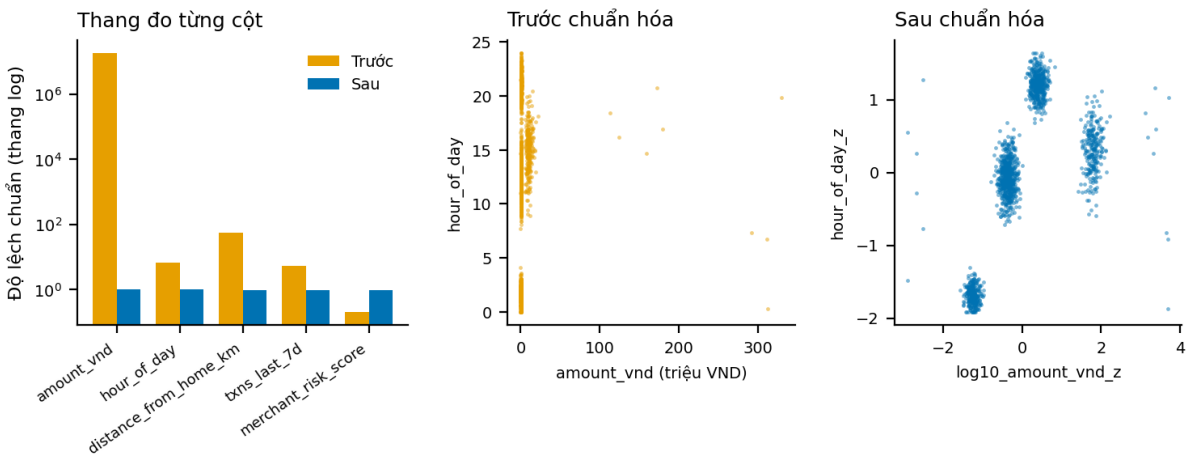
- Mã thuật toán: `src/clustering/`.
- Silhouette tự cài: `src/evaluation/metrics.py`.
- Dữ liệu demo: `data/raw/`.
- Các phương pháp scikit-learn dùng làm đối chứng: `baseline/`.

D - KẾT QUẢ, HÌNH ẢNH BỔ SUNG

Hình D.1 tập hợp một số tình huống biên dùng để quan sát điểm đúng trên ngưỡng Eps, điểm chuyển trạng thái và các dạng cập nhật phức tạp hơn. Hình D.2 minh họa sự khác nhau về thang đo thuộc tính trước và sau chuẩn hóa trên luồng thẻ.



Hình D.1: Một số trường hợp biên bổ sung của thao tác thêm và xóa điểm



Hình D.2: Thang đo đặc trưng luồng thẻ trước và sau chuẩn hóa